

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN

Rendimiento de MPI en un clúster de 96 núcleos

Multiplicación de matrices y regla del trapecio

Estudio experimental de comunicación, memoria y paralelismo

243 registros auditados, repeticiones individuales y análisis reproducible

Integrantes:

Paredes Malaga José Carlos
Jheeremy Manuel Alvarez Astete
Jose Alejandro Machaca Muñiz
David Tamo Turpo
Wilson Ramos

Curso:

Computación Paralela y Distribuida

Profesor:

Alvaro Henry Mamani Aliaga

Repositorio: <https://github.com/dtamotu/cluster-mpi-96-optimizaciones>

Arequipa, Perú – 25 de septiembre de 2026

Índice

Resumen	1
1. Preguntas, evidencia y diseño del estudio	2
1.1. Objetivo y preguntas de investigación	2
1.2. Inventario: qué son los 243 registros	2
1.3. Repeticiones, orden y estadística descriptiva	3
2. Plataforma y colocación de procesos	3
2.1. Qué especifica el comando de MPI	4
3. Algoritmos y límites del cronometraje	4
3.1. Multiplicación por franjas y difusión jerárquica	4
3.2. SUMMA 2D: qué implementación se evaluó	5
3.3. Trapecio y reparto ponderado	5
3.4. Tiempo interno, tiempo de pared y fases	5
3.5. Tráfico, unidades y comprobaciones	6
4. Resultados: antecedentes y baterías	6
4.1. Los 906 segundos: observación y límites de la explicación	6
4.2. Por qué las teselas ayudaron al cálculo local	7
4.3. Matrices en las baterías: la transición al segundo nodo	8
4.4. Trapecio: el tamaño cambia la conveniencia de distribuir	9
5. Resultados: campaña de optimización	10
5.1. Difusión jerárquica en la multiplicación completa	10
5.2. Microbenchmark: seleccionar la colectiva	11
5.3. Qué revelan los bytes de cada nodo	12
5.4. SUMMA 2D con el mismo número de procesos	13
5.5. Matrices mayores y búsqueda de un cruce	14
5.6. Trapecio simétrico y ponderado	15
5.7. El coste completo del procedimiento	16
6. Modelos que ayudan a interpretar los resultados	16
6.1. Cálculo, red y paralelismo útil	16
6.2. Memoria 1D: contar por nodo, no sumar la RAM del clúster	17
6.3. Qué aporta y qué cuesta la malla 2D	17
6.4. Qué optimizar después de reducir Bcast	18
7. Una corrida frente a repeticiones	19
7.1. Qué muestra la comparación	19
7.2. Consecuencias para la lectura del informe	20
8. Experimentos siguientes, ordenados por utilidad	20

8.1. Prioridad 1: comprobar knomial en la multiplicación completa	20
8.2. Prioridad 2: separar compilación y kernel local	21
8.3. Prioridad 3: encontrar el paralelismo local adecuado	21
8.4. Prioridad 4: una copia de B por nodo	21
8.5. Prioridad 5: 2D con trazas y una interfaz de entrada explícita	22
8.6. Prioridad 6: calibración P/E y estado de la red	22
8.7. Criterio de cierre de una nueva campaña	22
9. Auditoría, amenazas a la validez y procedencia	22
9.1. Comprobaciones repetibles	22
9.2. Fuentes de sesgo o incertidumbre	23
9.3. Problemas de procedencia detectados	24
10. Conclusiones	24
10.1. Respuesta al problema de rendimiento	24
10.2. Qué optimizaciones quedan respaldadas y cuáles abiertas	25
10.3. Alcance de la explicación y de la reproducibilidad	25
A. Reproducción, archivos y trazabilidad	26
A.1. Carpeta autocontenida y reconstrucción	26
A.2. Cómo seguir una cifra hasta su ejecución	26
A.3. Comando representativo de la campaña	27
B. Detalle de las baterías y antecedentes	27
B.1. Primera batería: trapecio	27
B.2. Primera batería: matrices y controles	28
B.3. Batería ampliada completa	29
B.4. Tráfico histórico y serie del mediodía	32
C. Detalle de la campaña repetida	32
D. Referencias	34

Resumen

Se estudia el rendimiento de dos cargas con distinta relación entre cálculo y comunicación: la multiplicación densa de matrices y la integración por la regla del trapecio. El entorno reúne cuatro equipos de 24 núcleos físicos conectados por Ethernet Gigabit y por Wi-Fi institucional. La evidencia principal comprende 50 ejecuciones de una primera batería, 75 de una ampliación y 118 de una campaña con tres o cinco repeticiones por configuración. Los antecedentes de madrugada se conservan aparte, porque emplearon otros programas y protocolos.

La hipótesis principal es que el aumento de procesos solo reduce el tiempo si el trabajo adicional de comunicación y coordinación permanece por debajo del cálculo que se ahorra. En matrices, un nodo con 24 procesos fue el mejor punto de las baterías, que probaron $P = 1, 24, 48, 72, 96$. En el trapecio con $n = 10^{11}$, las dos baterías alcanzaron aceleraciones de 68.56 y 68.20 respecto de un proceso al usar 96 procesos por Ethernet. Ningún resultado permite asumir que más núcleos siempre mejoran el rendimiento.

Para matrices de $N = 3072$, sustituir la difusión global de B por una difusión entre líderes de nodo seguida de una difusión local redujo la mediana con 96 procesos de 6.260 a 3.217 s en Ethernet y de 101.082 a 51.479 s en Wi-Fi. Las reducciones de tiempo fueron 48.6 % y 49.1 %; las de TX agregado, 52.5 % y 53.0 %. Las cinco repeticiones mantienen la dirección de la mejora. Aun así, esas medianas siguen por encima de los 0.649 y 0.666 s de la variante global con 24 procesos en un nodo.

En difusión aislada con 16 procesos, la selección `knomial` obtuvo medianas cercanas a las de la difusión jerárquica y un volumen agregado similar, pero concentró la transmisión en el servidor. SUMMA 2D, con 64 procesos, redujo las reservas de memoria del programa pero aumentó el tiempo y el tráfico en la configuración probada. El reparto ponderado del trapecio no produjo una mejora consistente. Estos resultados justifican estudiar la topología y el movimiento de datos antes de aumentar el paralelismo.

El informe reconstruye los límites del cronometraje, presenta todas las repeticiones relevantes, calcula memoria por nodo a partir del código y separa observación, explicación plausible y experimento pendiente. Las 243 filas principales se cotejaron con sus logs; las huellas y manifiestos se verificaron con el alcance detallado en la auditoría. Esta edición vuelve a analizar las ejecuciones conservadas: no incorpora nuevas mediciones del clúster.

Palabras clave: MPI, colectivas, afinidad, SUMMA, memoria distribuida, reproducibilidad, variabilidad.

El antecedente de 906.351 s. El 24 de septiembre de 2026 a las 00:58:05 se ejecutó una matriz de 3072×3072 por Wi-Fi con **ocho procesos: dos en servidor, dos en workers1, dos en workers2 y dos en workers4**. El máximo observado dentro de `MPI_Bcast(B)` fue 889.744 s. La distribución fue $2 + 2 + 2 + 2$; los ocho procesos no estaban concentrados en dos equipos.

1. Preguntas, evidencia y diseño del estudio

1.1. Objetivo y preguntas de investigación

El objetivo es explicar qué limita el tiempo de los programas disponibles y determinar qué cambios demostraron una mejora en este clúster. Se analizan cuatro preguntas: ¿cuándo compensa añadir nodos?, ¿qué parte de la comunicación se puede reducir?, ¿cómo influye la distribución de matrices en memoria? y ¿qué conclusiones resisten al observar las repeticiones individuales? El criterio principal es el tiempo total del algoritmo completo. El tiempo de una sola fase ayuda al diagnóstico, pero no sustituye ese criterio.

Se utiliza la separación entre partición, comunicación, agrupación y asignación de la metodología de Foster[1]. En matrices, partir filas resuelve la distribución del trabajo; replicar B determina buena parte del movimiento de datos; las teselas organizan la reutilización local; y el orden de rangos determina qué mensajes cruzan una interfaz física. Son decisiones diferentes. Una opción de transporte MPI no sustituye al diseño de esos cuatro niveles.

Tabla 1: Preguntas contrastables y evidencia disponible.

Pregunta	Comparación con controles compartidos	Alcance
¿Ayuda la jerarquía?	1D global frente a 1D jerárquica; mismo N , P , kernel, red y binario.	Cinco repeticiones, $N = 3072$, $P = 24, 96$.
¿Ayuda seleccionar Bcast?	Global, jerárquica y dos solicitudes tuned ; misma carga de difusión.	Tres repeticiones, $P = 16$, cuatro nodos.
¿Ayuda la partición 2D?	1D frente a 2D con el mismo número de procesos.	Tres repeticiones, $P = 64$, malla 8×8 .
¿Compensa un problema mayor?	1D global con 24 frente a 96 procesos.	$N = 3072, 6144, 7168$ por Ethernet.
¿Ayuda ponderar el trapecio?	Modos simétrico y ponderado del mismo programa corregido.	Tres repeticiones por tamaño y P .

1.2. Inventario: qué son los 243 registros

Tabla 2: Corpus principal. Una fila representa una ejecución, no una media.

Serie	Filas	Diseño
Primera batería	50	Dos redes, cinco valores de P , dos tamaños del trapecio y tres de matrices. Una ejecución por configuración.
Batería ampliada	75	40 ejecuciones Ethernet y 35 Wi-Fi. Añade matrices de 512, 4096 y 6144; el último tamaño solo por cable. Una ejecución por configuración.
Matrices principales	40	2 redes $\times 2$ valores de $P \times 2$ variantes 1D $\times 5$ repeticiones.
Difusión aislada	30	$2 \times 4 \times 3 = 24$ distribuidas, más $2 \times 3 = 6$ locales por Ethernet.
Comparación 1D/2D	12	2 redes $\times 2$ variantes $\times 3$ repeticiones, $P = 64$.
Matrices mayores	12	2 tamaños $\times 2$ valores de $P \times 3$ repeticiones, Ethernet.
Trapecio ponderado	24	2 tamaños $\times 2$ valores de $P \times 2$ repartos $\times 3$ repeticiones.
Total	243	$50 + 75 + (40 + 30 + 12 + 12 + 24)$.

Las 118 últimas ejecuciones pertenecen a la campaña de optimización. Las carpetas 20260925_074354 y 20260925_extra contienen 82 y 36 filas respectivamente. Las pruebas piloto y de humo no entran en las estadísticas. Los JSON históricos de caché, madrugada y mediodía se usan como antecedentes adicionales; no se suman a los 243 registros ni se mezclan con ellos para obtener

medianas.

1.3. Repeticiones, orden y estadística descriptiva

El lanzador agrupa las configuraciones por número de repetición y cambia su orden con una semilla reproducible: **20260925+rep** en la campaña principal y **20260926+rep** en la ampliación. Esto evita ejecutar siempre una variante primero. No elimina variaciones de señal, temperatura, frecuencia o carga ajena. Las dos baterías anteriores no ofrecen repetición dentro de cada configuración, por lo que sus puntos no tienen barras de incertidumbre estimables.

Para cada grupo se reportan mediana, mínimo, máximo y número de observaciones. Se conservan los valores extremos. El cociente principal es $R = \tilde{T}_A/\tilde{T}_B$; la reducción porcentual de B respecto de A es $100(1 - 1/R)$. Un factor 1.95 equivale aproximadamente a reducir el tiempo un 49 %, no un 95 %. El ahorro en segundos se calcula como diferencia de medianas.

Como comprobación descriptiva adicional se calcula $R_j = T_{A,j}/T_{B,j}$ para los casos del mismo bloque de repetición. No se supone que estas ejecuciones fueran simultáneas o inmediatamente consecutivas. La mediana de R_j y el cociente de medianas son estadísticos diferentes; se presentan con nombres distintos. Con tres o cinco observaciones no se declaran pruebas de significación ni intervalos de confianza. Que dos rangos se separen respalda la consistencia observada; no demuestra estabilidad en cualquier horario o red.

2. Plataforma y colocación de procesos

Tabla 3: Nodos participantes. El servidor también ejecuta trabajo numérico.

Nodo	Usuario	Ethernet	Wi-Fi
servidor	alumno16	10.7.50.202	10.7.134.117
workers1	alumno22	10.7.50.203	10.7.134.58
workers2	alumno21	10.7.50.201	10.7.134.59
workers4	alumno6	10.7.50.204	10.7.134.51

El inventario del proyecto describe un Intel Core Ultra 9 285 por equipo, con ocho núcleos P y dieciséis E, 24 núcleos físicos sin Hyper-Threading, 36 MB de caché L3 y 32 GB de RAM DDR5 en un módulo nominal de 5600 MT/s. Los cuatro nodos aportan 96 núcleos, pero su memoria se encuentra en espacios de direcciones distintos. El software documentado es Ubuntu 24.04 y Open MPI 4.1.6. **workers3** quedó excluido por incompatibilidad de MPI/PMIx.

Ethernet usa la interfaz **enp128s31f6**, un enlace Gigabit y la subred 10.7.50.0/24; Wi-Fi usa **wlp129s0f0** y 10.7.134.0/23. Ambas redes son compartidas con otros usuarios. El inventario no demuestra frecuencias, ocupación de canal o temperatura constantes. Tampoco se midió el ancho de banda sostenido de RAM en estas corridas. La velocidad nominal de un componente no se trata como una medida del ancho de banda obtenido por el programa.

Tabla 4: Distribución explícita de los procesos. Los ceros indican nodos no usados.

Caso	P	Servidor	W1	W2	W4
Antecedente de 906.351 s	8	2	2	2	2
Difusión aislada local	16	16	0	0	0
Difusión aislada distribuida	16	4	4	4	4
Referencia de las baterías	1	1	0	0	0
Un nodo completo	24	24	0	0	0
Dos nodos en baterías	48	24	24	0	0
Comparación 1D/2D	64	16	16	16	16
Tres nodos en baterías	72	24	24	24	0
Cuatro nodos completos	96	24	24	24	24

La campaña fija `-map-by slot -rank-by slot -bind-to core`. Con 96 procesos, los rangos 0–23 pertenecen al servidor, 24–47 a `workers1`, 48–71 a `workers2` y 72–95 a `workers4`. Se distinguen procesos MPI, hilos y unidades SIMD: los kernels del estudio no usan OpenMP; cada proceso calcula con un flujo de ejecución, mientras el compilador puede transformar sus bucles en instrucciones vectoriales. Una corrida con 96 procesos no mide un programa de un solo proceso con 96 hilos.

2.1. Qué especifica el comando de MPI

Los transportes se solicitan con `-mca btl self,vader,tcp`. `self` cubre mensajes al propio proceso, `vader` permite comunicación local por memoria compartida y `tcp` permite la comunicación IP entre equipos. Las opciones de interfaz limitan las redes que se deben usar. La FAQ de Open MPI 4.x indica que el transporte compartido local puede seleccionarse por defecto[3]; por tanto, la ausencia de la palabra `vader` en un comando antiguo no demuestra que sus mensajes locales atravesaran TCP.

Usar `vader` tampoco convierte automáticamente una difusión sobre todos los rangos en una difusión explícita entre líderes. Transporte, algoritmo colectivo y colocación son capas distintas. Para afirmar qué algoritmo eligió la biblioteca en una corrida histórica haría falta registrar esa selección. En este estudio se conocen el código invocado y las opciones solicitadas; la coincidencia de tiempo o tráfico no sustituye una traza interna.

3. Algoritmos y límites del cronometraje

3.1. Multiplicación por franjas y difusión jerárquica

Se calcula $C = AB$ con elementos `double`. La inicialización determinista es $A_i = 1 + 0.1(i \bmod 7)$ y $B_i = 2 - 0.1(i \bmod 5)$ sobre el índice lineal. El rango 0 reserva las tres matrices completas. En 1D, cada rango recibe $\lfloor N/P \rfloor$ filas de A y una fila adicional si su rango es menor que $N \bmod P$; reserva las filas correspondientes de C y una copia completa de B . Para $N = 3072$, $P = 96$, son 32 filas por proceso.

La secuencia global es `Scatterv(A)`, `Bcast(B)`, cálculo local y `Gatherv(C)`. En `1d-hier` se mantienen exactamente la partición, el kernel y la reunión. Solo cambia la difusión: primero participan los líderes de los nodos y después todos los procesos de cada nodo. El comunicador local se obtiene con `MPI_Comm_split_type` y `MPI_COMM_TYPE_SHARED`, que agrupa procesos capaces de compartir memoria[5]. Se selecciona el rango local 0 como líder. **Los buffers siguen siendo privados**; el programa no llama a `MPI_Win_allocate_shared`.

El kernel `tiléd` recorre teselas de lado 64 para favorecer la reutilización. No cambia la descomposición entre procesos. La alternativa `ikj` y la versión por bloques tienen complejidad aritmética $O(N^3)$; sus accesos efectivos a memoria y su rendimiento pueden ser muy distintos. La campaña usa `tiléd 64` en todas sus multiplicaciones.

3.2. SUMMA 2D: qué implementación se evaluó

Con $P = q^2$, cada rango pertenece a una fila y una columna de una malla. Su bloque tiene lado $b = N/q$. En cada paso k , se difunde A_{ik} por la fila y B_{kj} por la columna, y se acumula $C_{ij} += A_{ik}B_{kj}$. Esta organización sigue el esquema SUMMA[2]. La implementación evaluada exige $N \bmod q = 0$, usa colectivas bloqueantes y realiza las dos difusiones antes del cálculo: no implementa solapamiento entre comunicación y cálculo.

La entrada se encuentra centralizada. El rango 0 empaqueta todos los bloques de A y de B , los distribuye, y al final reúne y desempaqueta C . Ese trabajo pertenece al tiempo medido. Por eso no se compara con un algoritmo teórico que empieza con matrices ya distribuidas y termina sin recogerlas.

3.3. Trapecio y reparto ponderado

Se aproxima $\pi = \int_0^1 4/(1+x^2) dx$ mediante

$$Q_n = h \left[\frac{1}{2}f(0) + \sum_{i=1}^{n-1} f(ih) + \frac{1}{2}f(1) \right], \quad h = 1/n.$$

Cada proceso acumula un intervalo de índices y una reducción suma un `double` por rango. El volumen útil es pequeño frente al cálculo cuando $n = 10^{11}$. Las métricas se resumen con reducciones adicionales después del intervalo medido; los contadores de interfaz pueden incluirlas.

El programa de la campaña incluye explícitamente el extremo $f(1)/2$ en el último rango. Sus dos modos usan ese mismo cálculo corregido. El modo ponderado asigna peso 1.2658 a los primeros ocho rangos de cada bloque de 24 y peso 1 a los dieciséis restantes. El código usa `rank%24`; no consulta el tipo físico de núcleo. La interpretación como reparto P/E requiere verificar la correspondencia entre rangos y tipos de núcleo. Los logs de afinidad ayudan a reconstruirla, pero el algoritmo no la detecta por sí mismo.

3.4. Tiempo interno, tiempo de pared y fases

Tabla 5: Operaciones incluidas y excluidas en el tiempo interno de matrices.

Etapa	Relación con <code>T_Total</code>
Antes del cronómetro	<code>MPI_Init</code> , reservas, inicialización de A/B , creación de comunicadores y barrera inicial.
Dentro, variante 1D	Distribución de A , difusión de B , cálculo, reunión de C y barrera final.
Dentro, variante 2D	Empaquetamiento y reparto inicial, pasos SUMMA, cálculo, reunión, desempaquetamiento y barrera final.
Después del cronómetro	Liberaciones, reducciones de métricas, comprobaciones numéricas, salida y <code>MPI_Finalize</code> .

Cada rango mide duraciones con `MPI_Wtime`; se reporta el máximo entre rangos. La creación de comunicadores jerárquicos ocurre antes del cronómetro, de modo que su coste no se incluye en la comparación interna. La columna `pared_s`, medida por Python, abarca además comprobaciones

de memoria para matrices grandes, lectura remota de contadores, lanzamiento, inicialización, comprobaciones finales y cierre. Representa el coste del procedimiento de prueba; no es un cronómetro puro del comando `mpirun`.

Las fases se reducen por separado. Para tiempos locales $t_{r,k}$,

$$\max_r \sum_k t_{r,k} \leq \sum_k \max_r t_{r,k}.$$

La suma de máximos mezcla procesos diferentes. Además, un proceso puede entrar a **Gather** mientras otro sigue en **Bcast**; parte del tiempo de la colectiva es espera. Por ello no se apilan los máximos como porcentajes de un único total, ni se interpreta su suma como tiempo de red. La barrera final también aporta tiempo al total de matrices. El trapecio y el microbenchmark tienen límites propios y no usan necesariamente esa barrera final.

3.5. Tráfico, unidades y comprobaciones

La campaña lee contadores TX y RX antes y después en cada nodo participante. Las consultas son secuenciales mediante SSH; los intervalos no están sincronizados exactamente. El tráfico de control y de aplicaciones ajenas puede contribuir. Se utiliza TX agregado para comparar configuraciones: sumar TX y RX entre todos los nodos contaría ambas observaciones de muchos mensajes. En la batería ampliada solo se conserva TX+RX del servidor; no se convierte esa cifra en TX agregado del clúster.

En este informe, MB significa 10^6 bytes y GiB significa 2^{30} bytes. Una matriz pesa $M = 8N^2$ bytes: para $N = 3072$, $M = 75.497472$ MB. Las cifras de interfaces no representan directamente carga útil MPI, tráfico radiado o retransmisiones TCP identificadas.

Las matrices se comprueban mediante suma diagonal y suma ponderada, y la difusión mediante una huella de 64 bits idéntica entre procesos. Son controles útiles de consistencia, con precisión limitada por la impresión en los logs; no sustituyen una comparación elemento por elemento con una referencia independiente. El trapecio se compara con π y la campaña rechaza errores mayores que 10^{-8} . La auditoría del anexo comprueba la coherencia entre archivos sin convertir estos controles en una demostración general de corrección.

4. Resultados: antecedentes y baterías

4.1. Los 906 segundos: observación y límites de la explicación

El log `datos/historicos/repetir_profiling_cluster.log` registra el inicio del caso central el 24 de septiembre a las 00:58:05. La configuración fue $N = 3072$, ocho procesos y cuatro nodos por Wi-Fi. El comando conservado en el script es:

```
1 mpirun -H servidor:2,workers1:2,workers2:2,workers4:2 \
2   -np 8 \
3   --mca btl_tcp_if_include 10.7.134.0/23 \
4   --mca oob_tcp_if_include 10.7.134.0/23 \
5   /tmp/mpi_matrix_profiling 3072
```

El tiempo total fue 906.351077 s; el máximo de difusión de B , 889.744162 s; el máximo de cálculo, 1.735076 s. El cociente entre difusión y total es 98.17 %. Indica que un rango pasó casi todo el intervalo observado dentro de esa llamada, incluyendo sus esperas. No es un reparto aditivo de todo el tiempo entre fases. El máximo de reunión fue también muy alto, 885.556909 s: sumarlo al de difusión produciría una interpretación incorrecta.

Tabla 6: Serie original por Wi-Fi con dos repeticiones. $P = 4, 8, 16$ usa respectivamente 1, 2 y 4 procesos en cada uno de los cuatro nodos. Todos los casos son de $N = 3072$.

P	Rep.	Inicio	T_{Bcast} (s)	T_{Calc} (s)	T_{Total} (s)
4	1	00:53:08	276.244	2.645	290.104
4	2	01:24:59	87.828	2.448	101.805
8	1	00:58:05	889.744	1.735	906.351
8	2	01:26:48	306.769	1.737	322.513
16	1	01:13:19	289.159	0.786	300.886
16	2	01:32:18	294.318	0.964	308.340

Con los mismos ocho procesos, la segunda ejecución tardó 322.513 s y el cálculo permaneció alrededor de 1.737 s. La variación principal se produjo fuera del cálculo. Esto respalda estudiar las comunicaciones; no identifica por sí solo interferencia, retransmisiones o el algoritmo interno responsable. No se conservaron contadores TCP ni de radio que permitan decidir entre esas causas.

Las cifras de aproximadamente 1112 MB para 1D y 423 MB para 2D pertenecen a otra serie con 16 procesos, cuatro por nodo. En esa serie, la mediana de 1D fue 330.818 s y la de 2D, 122.928 s, con valores 2D de 66.172 y 179.684 s. La dispersión y el protocolo distinto impiden trasladar ese factor de mejora a la campaña posterior. Las tablas históricas completas se conservan en el anexo.

4.2. Por qué las teselas ayudaron al cálculo local

El antecedente de un solo equipo compara `ikj`, `tilted 1D` y `tilted 2D` con $N = 3072$. Son medianas de tres repeticiones históricas, no ejecuciones de la campaña de difusión:

Tabla 7: Antecedente local: tiempos totales en segundos. Los casos 2D requieren un número cuadrado de procesos.

P	1D-ikj (s)	1D-tilted (s)	2D-tilted (s)
1	7.150618	3.934298	3.927859
4	5.118120	1.101255	1.105275
8	5.297588	0.589795	–
16	5.483845	0.474050	0.381260
24	5.571556	0.503265	–

Con un proceso, 1D pasa de 7.151 a 3.934 s al usar teselas, un factor aproximado de 1.82. Con 24 procesos, pasa de 5.572 a 0.503 s, un factor 11.07. Esto muestra que la organización local puede cambiar el resultado aun con la misma complejidad cúbica. Es coherente con mayor reutilización de datos, pero no mide fallos de caché ni bytes de DRAM. Una cuenta basada en referencias del bucle, como $8N^3 \simeq 231.93$ GB para $N = 3072$, describe accesos lógicos posibles; no es una medición de tráfico de memoria.

Este antecedente también impide declarar 24 procesos como óptimo universal: con 1D tiled, 16 procesos dieron 0.474 s frente a 0.503 s con 24; con 2D y 16 procesos se observaron 0.381 s. El barrido fino por número de procesos necesita repetirse con el binario actual y

condiciones compartidas. La afirmación válida para las baterías es que 24 fue el mejor punto entre 1, 24, 48, 72, 96.

4.3. Matrices en las baterías: la transición al segundo nodo

Tabla 8: Primera batería, $N = 3072$. $S_P = T_1/T_P$ se calcula dentro de cada red. Una corrida por celda.

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	3.816321	3.904030	1.00	1.00	1.02
24	0.512022	0.553372	7.45	7.05	1.08
48	2.815416	44.256782	1.36	0.09	15.72
72	4.352181	65.984071	0.88	0.06	15.16
96	6.174707	101.042337	0.62	0.04	16.36

El paso de uno a 24 procesos reduce el tiempo Ethernet de 3.816 a 0.512 s. El paso de 24 a 48 procesos, que añade el primer nodo remoto, lo aumenta a 2.815 s. Con 96 procesos llega a 6.175 s y queda incluso por encima de la referencia de un proceso. Por Wi-Fi la penalización es mayor: 101.042 s con 96 procesos frente a 0.553 s con 24.

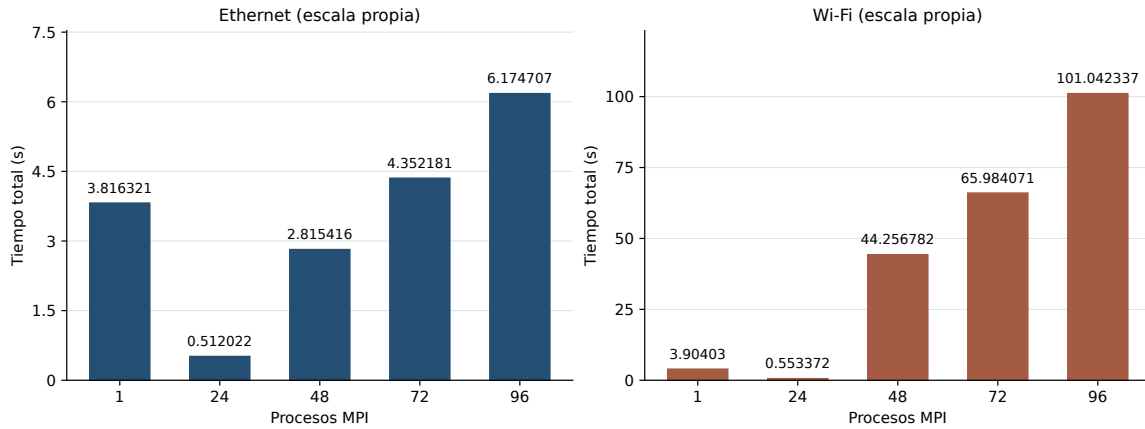


Figura 1: Primera batería: tiempos para los tres tamaños de matriz. Cada punto corresponde a una ejecución. Las tendencias entre redes se comparan dentro del mismo tamaño y distribución de procesos.

La segunda batería confirma el patrón para seis tamaños por Ethernet y cinco por Wi-Fi. Con $N = 6144$, un nodo completo tarda 3.839 s y cuatro nodos 25.037 s. El cociente 6.52 procede de esas dos corridas únicas, no de una mediana repetida. El crecimiento de N mejora la relación entre trabajo aritmético y datos, pero no elimina la penalización distribuida en esta batería.

Tabla 9: Batería ampliada: tiempos totales de matrices, en segundos. Una ejecución por configuración; el guion indica que no se ejecutó esa prueba.

N	Cable T_1	Cable T_{24}	Cable T_{96}	Wi-Fi T_{24}	Wi-Fi T_{96}
512	0.019	0.007	0.211	0.007	4.299
1024	0.145	0.034	0.760	0.035	13.648
2048	1.198	0.200	2.787	0.215	47.759
3072	3.889	0.547	6.148	0.553	99.945
4096	9.914	1.325	11.042	1.410	178.957
6144	32.888	3.839	25.037	—	—

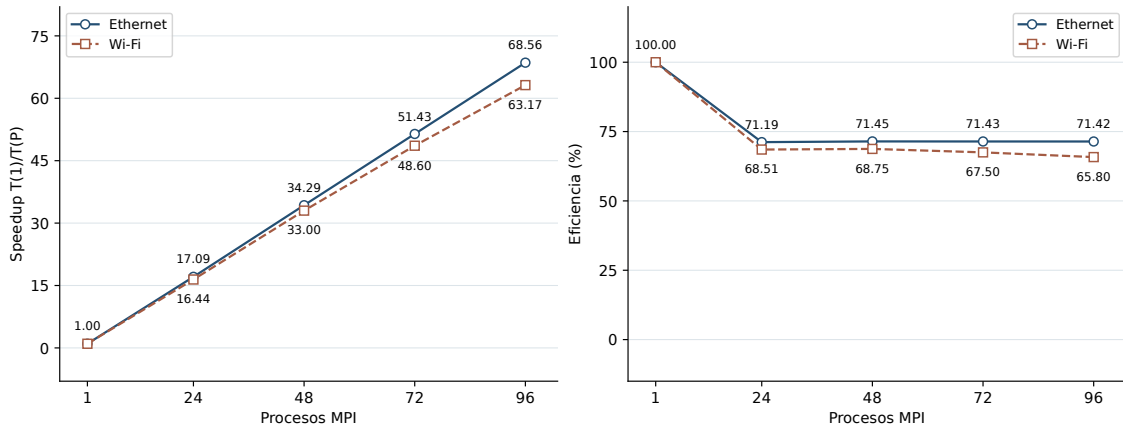
4.4. Trapecio: el tamaño cambia la conveniencia de distribuir

Tabla 10: Batería ampliada del trapecio. Las aceleraciones se refieren al proceso único de la misma red.

n	Red	T_1 (s)	T_{24} (s)	T_{96} (s)	S_{96}
10^8	Cable	0.066984	0.003921	0.001508	44.42
10^8	Wi-Fi	0.067826	0.005612	0.115863	0.59
10^{11}	Cable	66.488999	3.842393	0.974949	68.20
10^{11}	Wi-Fi	66.857550	3.937623	1.135211	58.89

Para $n = 10^{11}$, Ethernet pasa de 66.489 a 0.975 s con 96 procesos: $S_{96} = 68.20$ y eficiencia $S_{96}/96 = 71.0$ %. La primera batería obtuvo $S_{96} = 68.56$. Estos valores describen aceleración interna respecto del mismo programa MPI con un proceso. No son una medida de eficiencia energética ni una comparación con una implementación secuencial distinta.

Para $n = 10^8$, la batería ampliada pasa por Wi-Fi de 0.067826 s con un proceso a 0.115863 s con 96. La comunicación y coordinación pueden costar más que el trabajo ahorrado, aun cuando solo se reduce un escalar útil. Un mensaje pequeño necesita menos bytes, pero no tiene coste temporal nulo. La eficiencia tampoco debe interpretarse suponiendo que todos los núcleos tienen idéntico rendimiento: los equipos combinan núcleos P y E, y la asignación y frecuencia importan.

**Figura 2:** Primera batería del trapecio: métricas de escalamiento. Las mediciones únicas describen la tendencia; la campaña repetida posterior usa otro programa y se analiza aparte.

5. Resultados: campaña de optimización

5.1. Difusión jerárquica en la multiplicación completa

La comparación principal conserva $N = 3072$, kernel tiled 64, proceso por núcleo y el mismo binario. La variante global usa una difusión sobre todos los rangos; la jerárquica separa líderes y grupos locales. La figura muestra cada observación y evita que la mediana oculte la dispersión.

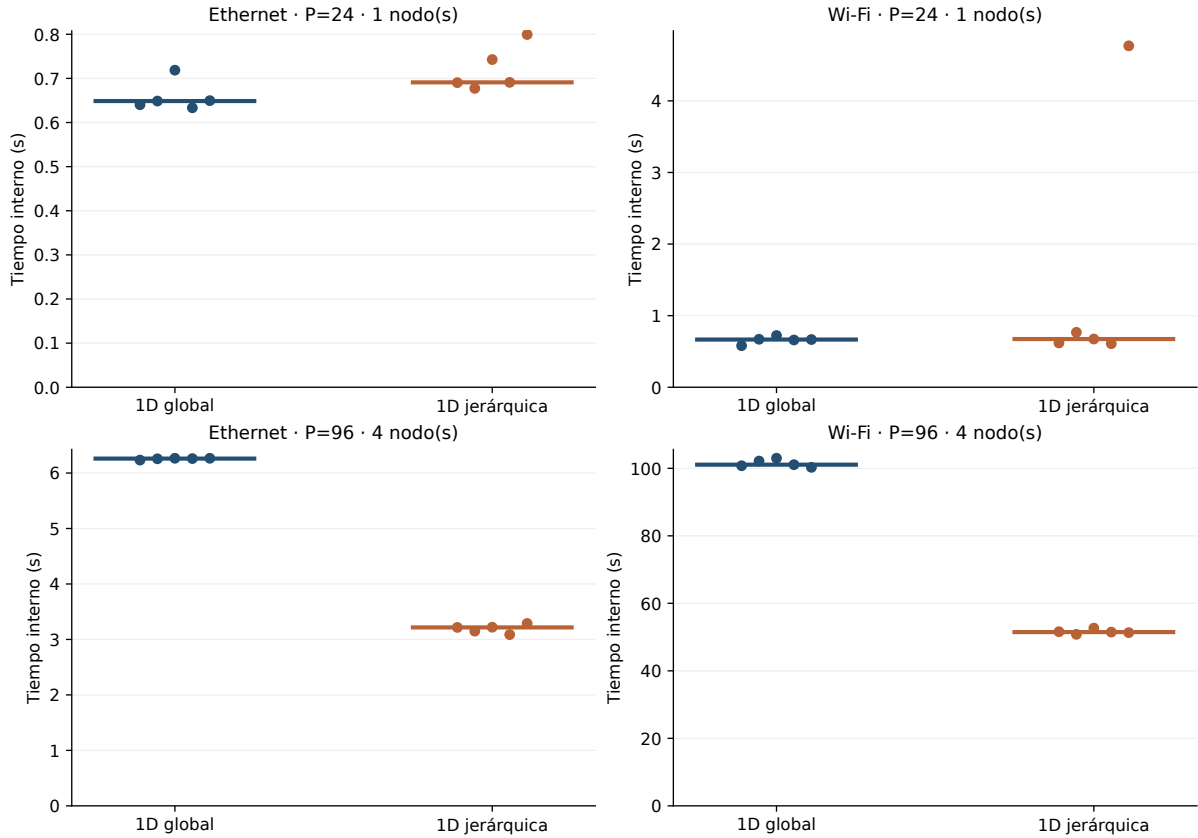


Figura 3: Las 40 observaciones principales de matrices. Cada punto es una corrida; el segmento horizontal es la mediana de cinco. Los paneles tienen escalas verticales propias. Se mantiene el valor de 4.769 s del caso local jerárquico bajo la etiqueta Wi-Fi.

Con 96 procesos, Ethernet pasa de una mediana de 6.260 s (rango 6.233–6.266) a 3.217 s (3.088–3.288). Wi-Fi pasa de 101.082 s (100.293–102.991) a 51.479 s (50.817–52.659). Los rangos están separados. Además, todas las razones dentro de los bloques de repetición favorecen la jerarquía: entre 1.906 y 2.027 por Ethernet y entre 1.953 y 2.010 por Wi-Fi.

Con 24 procesos locales, las medianas son próximas y los rangos se solapan. En Ethernet son 0.649 s global y 0.691 s jerárquico; en los lanzamientos etiquetados Wi-Fi, 0.666 y 0.673 s. Como los datos permanecen en un único equipo, esa etiqueta no implica que las matrices viajen por radio. El valor extremo de 4.769 s en la variante jerárquica local se conserva; su causa no está determinada. Excluirlo por resultar incómodo sesgaría el informe.

Tabla 11: Medianas de los máximos por fase con 96 procesos, en segundos. Cada columna se resume por separado; no se debe sumar para reconstruir el total.

Red	Variante	Scatter	Bcast	Cálculo	Gather	Total
Ethernet	1d	0.532	5.611	0.156	3.191	6.260
Ethernet	1d-hier	0.624	2.474	0.161	1.634	3.217
Wi-Fi	1d	11.373	88.904	0.153	61.103	101.082
Wi-Fi	1d-hier	10.534	40.272	0.151	30.750	51.479

El cálculo cambia poco entre variantes, mientras la difusión disminuye claramente. Esto concuerda con la intervención realizada en el código. La disminución de **Gather** no prueba que se haya optimizado esa llamada: su código no cambió y su tiempo puede contener espera. La comparación de fases identifica dónde aparecen duraciones grandes, pero no proporciona una descomposición exclusiva del tiempo ahorrado.

Tabla 12: Magnitud de los efectos. A/B nombra el caso de referencia y su alternativa; las reducciones se calculan a partir de medianas. Mismo $N = 3072$ y cuatro nodos. Un porcentaje negativo indica un aumento de TX.

Red y P	A / B	$\tilde{T}_A/\tilde{T}_B$	Ahorro (s)	Tiempo ↓	TX ↓
Ethernet 96	1d / 1d-hier	1.95	3.043	48.6 %	52.5 %
Ethernet 16	world / hier	2.11	2.260	52.6 %	76.9 %
Ethernet 16	hier / tuned-knomial	1.03	0.057	2.8 %	-0.0 %
Ethernet 64	2d / 1d	1.74	2.344	42.7 %	47.2 %
Wi-Fi 96	1d / 1d-hier	1.96	49.604	49.1 %	53.0 %
Wi-Fi 16	world / hier	4.37	102.579	77.1 %	77.3 %
Wi-Fi 16	hier / tuned-knomial	1.03	0.888	2.9 %	-0.3 %
Wi-Fi 64	2d / 1d	1.69	35.586	40.8 %	49.4 %

La mejora distribuida no alcanza al nodo local: con Ethernet, 3.217/0.649 es aproximadamente 4.96; con Wi-Fi, 51.479/0.666 es 77.26. Estos cocientes comparan variantes y números de procesos diferentes y sirven para decidir cuál configuración observada terminó antes. No representan el efecto aislado de añadir una sola opción de MPI.

5.2. Microbenchmark: seleccionar la colectiva

Se difunden 3072^2 dobles, equivalentes a 75.497 MB, con 16 procesos. En cuatro nodos se colocan cuatro procesos por nodo; el control local coloca los dieciséis en el servidor. Las variantes **tuned-knomial** y **tuned-scatter** solicitan al componente **tuned** los identificadores 7 y 8 mediante `coll_tuned_use_dynamic_rules=1`. La correspondencia de nombres se verifica en el código de Open MPI 4.1.6[4].

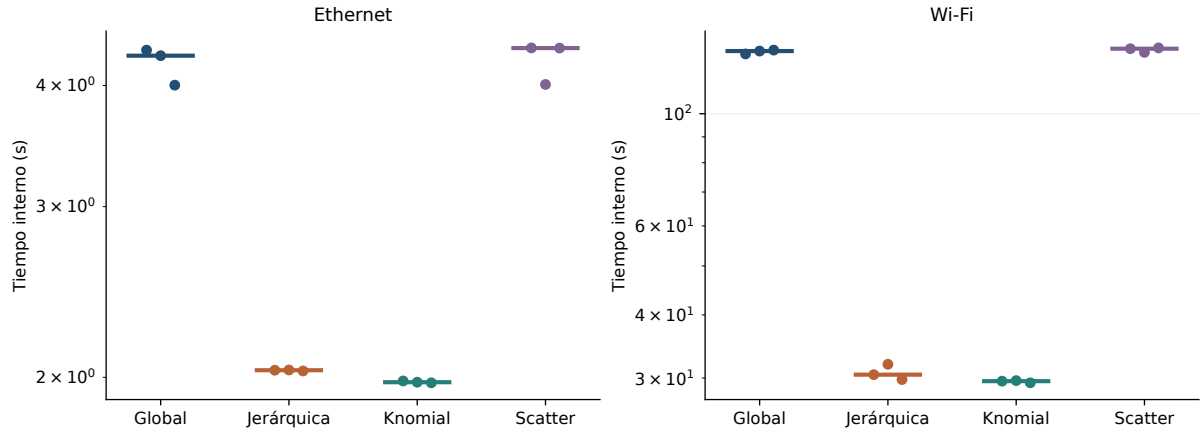


Figura 4: Difusión aislada con 16 procesos distribuidos $4 + 4 + 4 + 4$. Tres observaciones por variante; segmentos: medianas. Escala logarítmica, indicada en el eje.

En Ethernet, las medianas global, jerárquica y knomial son 4.294, 2.034 y 1.977 s. En Wi-Fi son 133.043, 30.464 y 29.575 s. La solicitud scatter/allgather dio 4.372 y 134.479 s, próxima al caso global. Esa similitud no prueba que el caso global eligiera internamente ese algoritmo; falta la traza de selección.

Knomial reduce la mediana frente a la jerárquica en 0.057 s por Ethernet y 0.888 s por Wi-Fi, aproximadamente 3 %. Las tres observaciones conservan la dirección de la diferencia. La mejora más importante es la que separa las variantes de bajo tráfico de la global, especialmente en Wi-Fi. No se midió todavía esta selección dentro de la multiplicación completa con 96 procesos: extrapolar directamente el factor del microbenchmark sería incorrecto.

5.3. Qué revelan los bytes de cada nodo

Tabla 13: Medianas de TX por nodo, en MB. M96: multiplicación completa con 96 procesos; B16: difusión aislada con 16. Total: mediana de la suma por corrida, que puede diferir de la suma de las cuatro medianas.

Red	Prueba / variante	Servidor	W1	W2	W4	Total
Ethernet	M96 1d	455.7	20.8	258.0	21.1	755.5
Ethernet	M96 1d-hier	139.1	99.6	99.6	20.3	358.9
Ethernet	B16 world	297.8	238.3	258.1	238.3	1032.5
Ethernet	B16 hier	79.3	79.4	79.4	0.3	238.4
Ethernet	B16 tuned-knomial	237.5	0.3	0.3	0.3	238.4
Wi-Fi	M96 1d	439.4	20.9	248.3	20.8	729.3
Wi-Fi	M96 1d-hier	133.1	95.2	95.2	19.5	342.9
Wi-Fi	B16 world	289.7	231.6	252.6	231.9	1006.5
Wi-Fi	B16 hier	75.7	76.0	76.0	0.3	228.0
Wi-Fi	B16 tuned-knomial	227.3	0.4	0.4	0.4	228.6

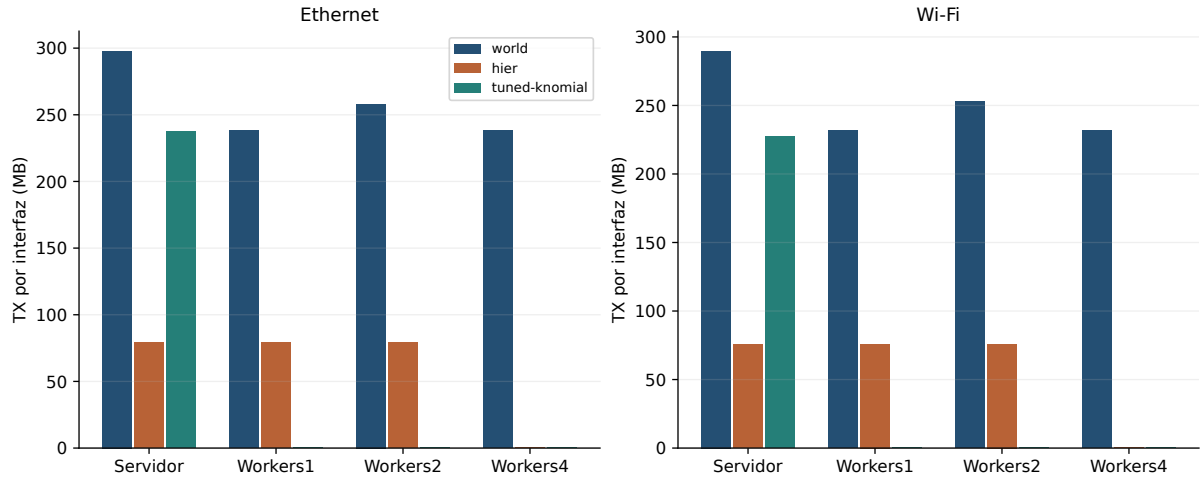


Figura 5: Distribución de TX en la difusión aislada. El volumen agregado parecido de jerárquica y knomial oculta un reparto distinto de la transmisión. Son contadores de interfaz, no una traza de cada mensaje MPI.

En Ethernet, la jerárquica registra alrededor de 79 MB en cada uno de los tres primeros nodos y casi cero en el cuarto. Knomial registra unos 238 MB en el servidor y menos de 1 MB en cada worker. El patrón Wi-Fi es semejante. La observación respalda que el reparto de transmisión cambió; identificar todos los saltos requeriría una traza de comunicaciones. En particular, total similar no significa mismo árbol ni misma presión sobre cada interfaz.

Como referencia ideal de carga útil, hacer llegar una copia de B a cada uno de tres nodos remotos exige $3M = 226.492$ MB de transferencias entre nodos si cada copia cruza una sola vez. Los 228–238 MB observados en las variantes de bajo tráfico son próximos a esa referencia. La diferencia no se convierte en una tasa de retransmisiones: los contadores incluyen cabeceras, control y otros posibles tráficos. En la multiplicación completa se añaden reparto de A y reunión de C , de modo que su TX no se compara directamente con el de Bcast aislado.

5.4. SUMMA 2D con el mismo número de procesos

Con 64 procesos, Ethernet dio 3.147 s para 1D y 5.492 s para 2D; Wi-Fi, 51.728 y 87.314 s. La partición 2D fue 1.74 y 1.69 veces más lenta según el cociente de medianas sin redondear. TX pasó de 358.0 a 678.1 MB por Ethernet y de 343.0 a 677.7 MB por Wi-Fi. Las tres razones por bloque favorecen 1D. No se compara 2D con 96 procesos, porque 96 no cumple la malla cuadrada que exige esta implementación.

El resultado limita una hipótesis concreta: disminuir memoria por proceso no garantizó disminuir comunicación física ni tiempo completo con cuatro nodos y este tamaño. El análisis de la malla, la centralización inicial y los ocho pasos de difusión se desarrolla en la sección 6. No se concluye que SUMMA sea inferior para otras topologías, entradas distribuidas o implementaciones con solapamiento.

5.5. Matrices mayores y búsqueda de un cruce

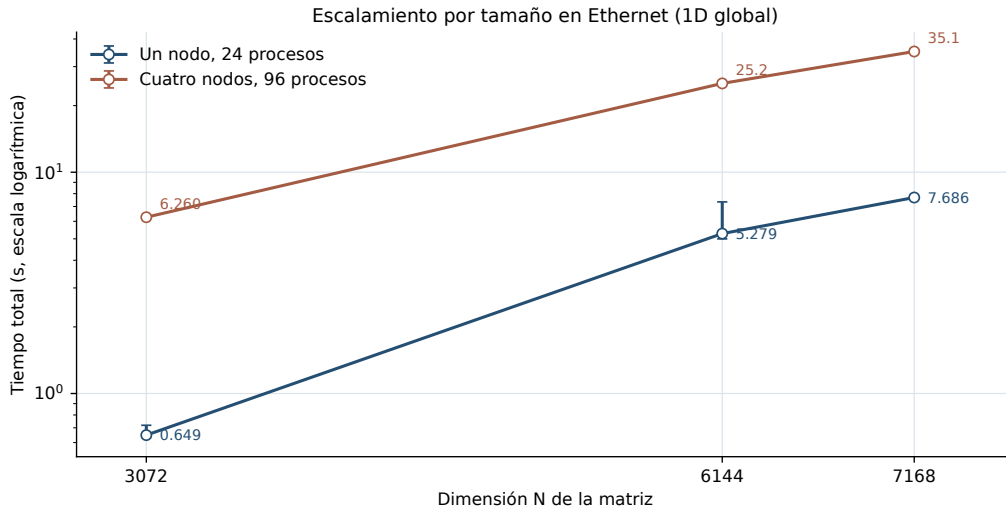


Figura 6: 1D global por Ethernet. Medianas con mínimo y máximo: cinco repeticiones para $N = 3072$ y tres para $N = 6144, 7168$.

Con $N = 6144$, las medianas son 5.279 s para 24 procesos y 25.194 s para 96; con $N = 7168$, 7.686 y 35.116 s. El cociente T_{96}/T_{24} baja de 9.65 a 4.77 y 4.57 al aumentar N . El patrón es compatible con un coste de cálculo que crece cúbicamente frente a buffers y transferencia que crecen cuadráticamente, pero tres tamaños no identifican un modelo predictivo fiable.

No se observó un tamaño en el cual la versión distribuida 1D global superara al nodo local. Tampoco se ejecutó 1D jerárquica en los dos tamaños mayores. Por eso no se atribuye el mismo comportamiento a esa variante y no se extrapola un tamaño de cruce a partir de una recta. El modelo de memoria de la siguiente sección aporta otra restricción concreta.

5.6. Trapecio simétrico y ponderado

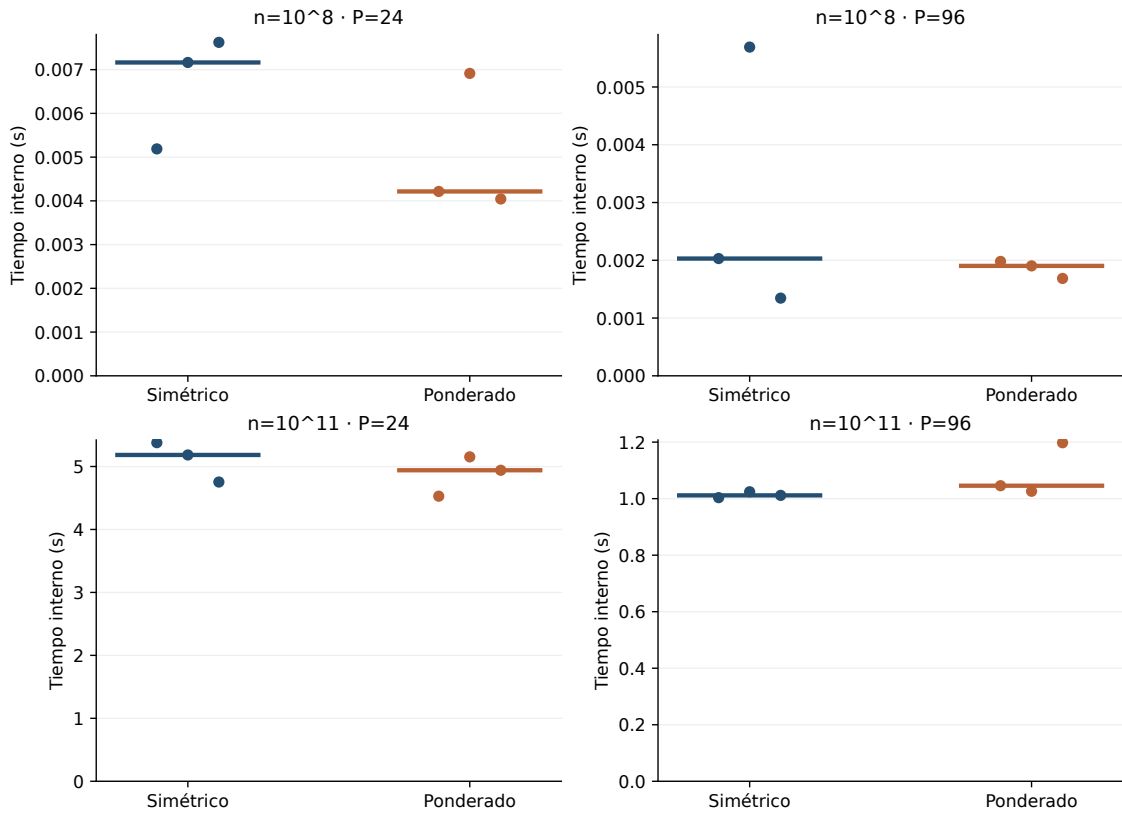


Figura 7: Las 24 observaciones del trapecio por Ethernet. Tres repeticiones por variante, tamaño y número de procesos. Cada panel tiene su propia escala; los tiempos pequeños se mantienen en segundos.

Tabla 14: Tiempo y desbalance observados en cada repetición. Desbalance = máximo/mínimo del cálculo entre rangos.

n, P	Reparto	Rep. 1		Rep. 2		Rep. 3	
		T (s)	Desb.	T (s)	Desb.	T (s)	Desb.
$10^8, 24$	simétrico	0.0052	1.625	0.0072	2.183	0.0076	2.353
	asimétrico	0.0042	1.100	0.0069	1.863	0.0040	1.100
$10^8, 96$	simétrico	0.0020	1.333	0.0057	1.283	0.0013	1.275
	asimétrico	0.0020	1.195	0.0019	1.142	0.0017	1.278
$10^{11}, 24$	simétrico	5.379	1.247	5.183	1.244	4.753	1.286
	asimétrico	4.528	1.272	5.153	1.396	4.940	1.233
$10^{11}, 96$	simétrico	1.004	1.263	1.024	1.290	1.012	1.272
	asimétrico	1.046	1.188	1.026	1.165	1.198	1.360

Para $n = 10^{11}$ y 24 procesos, la mediana baja de 5.183 a 4.940 s, un 4.7 %. Los rangos se solapan: la tercera corrida simétrica es más rápida que dos ponderadas. Con 96 procesos, la mediana ponderada es 1.046 s frente a 1.012 s simétrica, un 3.4 % mayor; las tres comparaciones dentro del bloque favorecen el reparto simétrico. El peso 1.2658 no queda validado como una mejora general.

El desbalance tampoco resume por sí solo el objetivo. Con 96 procesos, el reparto ponderado reduce el cociente máximo/mínimo en las dos primeras repeticiones, pero sus tiempos totales son mayores. Acercar duraciones puede ocurrir porque se ralentizó a un proceso antes rápido. La optimización debe minimizar el máximo absoluto y el total, no solamente igualar cocientes. Las salidas individuales de la calibración exploratoria del peso no se conservaron, por lo que no se utiliza como evidencia cuantitativa.

5.7. El coste completo del procedimiento

Tabla 15: Medianas del tiempo interno y del procedimiento de prueba. Δ es la mediana de las diferencias por corrida; no necesariamente equivale a restar las dos columnas de medianas.

Prueba / tamaño	Red	P	Variante	Interno (s)	Pared (s)	Δ (s)
M3072	Ethernet	24	1d	0.648645	1.836	1.188
M3072	Ethernet	96	1d	6.260268	9.275	3.021
M3072	Ethernet	96	1d-hier	3.216833	6.434	3.146
M3072	Wi-Fi	96	1d	101.082387	105.299	4.520
M3072	Wi-Fi	96	1d-hier	51.478540	55.935	4.802
$n = 10^8$	Ethernet	24	simetrico	0.007164	0.998	0.990
$n = 10^8$	Ethernet	96	simetrico	0.002030	2.918	2.916
$n = 10^{11}$	Ethernet	24	simetrico	5.183325	6.115	0.952
$n = 10^{11}$	Ethernet	96	simetrico	1.011597	4.301	3.297

Para matrices distribuidas por Ethernet, la jerarquía reduce aproximadamente a la mitad el tiempo interno, pero el tiempo de pared pasa de 9.275 a 6.434 s: una reducción cercana al 31 %. Los costes externos al algoritmo siguen presentes. En Wi-Fi, el cómputo distribuido tarda lo suficiente para que estos costes representen una fracción menor.

El caso más claro es el trapecio pequeño: 96 procesos mejoran la mediana interna de 7.164 a 2.030 ms respecto de 24, pero el procedimiento tarda 2.918 s frente a 0.998 s. Para una operación aislada, la elección que gana dentro del cronómetro puede hacer esperar más al usuario. En un servicio con procesos persistentes y muchas integraciones se podría amortizar el lanzamiento, pero eso sería otro experimento. Esta distinción es necesaria antes de recomendar una configuración para uso real.

6. Modelos que ayudan a interpretar los resultados

6.1. Cálculo, red y paralelismo útil

Una multiplicación densa requiere aproximadamente $2N^3$ operaciones aritméticas. Dividir filas reparte ese trabajo, pero cada rango sigue necesitando todo B . Un modelo orientativo es

$$T(N, P, H) \approx T_{\text{reparto}} + T_{\text{difusion}} + T_{\text{calculo}} + T_{\text{reunion}} + T_{\text{espera}},$$

donde H es el número de nodos. La expresión es un esquema de costes del camino crítico, no una instrucción para sumar los máximos impresos en los logs. Para mensajes de tamaño m , el modelo elemental $\alpha + \beta m$ separa latencia de coste por byte. En una colectiva intervienen varios mensajes, decisiones de algoritmo y posibles cuellos de botella compartidos; una sola α o β no explica todo el clúster.

En $N = 3072$ hay unos 57.98 mil millones de operaciones. La mediana global local de 0.649 s corresponde aproximadamente a 89.4 GFLOP/s efectivos para la multiplicación completa; la jerárquica con 96 procesos y Ethernet, a 18.0 GFLOP/s. Se usa $2N^3/T_{Total}$ y se incluye distribución y reunión. Estas cifras no miden el pico de las CPU ni la tasa de su kernel aislado. El aumento de capacidad aritmética disponible no compensa el aumento de tiempo total.

El trapecio necesita mucho menos intercambio por cantidad de trabajo. Al pasar de 10^8 a 10^{11} evaluaciones, crece el cálculo por un factor mil sin multiplicar por mil el escalar final reducido. Por eso la misma infraestructura presenta un comportamiento opuesto al de matrices. Una ley de aceleración basada solo en una fracción serial constante sería insuficiente para matrices: aquí el coste de comunicación cambia con P , con el número de nodos y con la colectiva elegida.

6.2. Memoria 1D: contar por nodo, no sumar la RAM del clúster

Sea $M = 8N^2$ bytes, $r = P/H$ procesos por nodo y s_h la suma de filas asignadas a los procesos del nodo h . Las reservas grandes explícitas de la variante 1D son:

$$\mathcal{M}_h = rM + 16Ns_h + 2M \mathbf{1}_{h=0}.$$

El primer término corresponde a una copia de B por proceso; el segundo a las porciones locales de A y C ; el último a las matrices globales A y C del servidor. Su copia global de B ya se cuenta entre las r copias. Si las filas quedan equilibradas por nodo, $s_h \simeq N/H$ y

$$\mathcal{M}_0 \simeq (r + 2/H + 2)M, \quad \mathcal{M}_{h>0} \simeq (r + 2/H)M.$$

Tabla 16: Reservas explícitas de matrices calculadas desde el código 1D, tanto global como jerárquico. La tabla considera el reparto exacto de filas, incluso si N no es divisible por P .

N	P	Nodos	Una matriz (MB)	Servidor (GiB)	Worker máx. (GiB)
3072	24	1	75.497	1.969	–
3072	96	4	75.497	1.863	1.723
6144	24	1	301.990	7.875	–
6144	96	4	301.990	7.453	6.891
7168	24	1	411.042	10.719	–
7168	96	4	411.042	10.145	9.380

Los valores son un modelo de buffers reservados, no medidas de RSS o de memoria pico física. Excluyen buffers internos de MPI, biblioteca, sistema y otros programas; algunas páginas pueden materializarse al primer acceso. Para $N = 7168$, el nodo local reserva unos 10.719 GiB en matrices, y el servidor de la versión de cuatro nodos, unos 10.145 GiB. La disminución es pequeña porque ambos mantienen 24 copias completas de B en ese equipo. Sumar 4×32 GB no describe el tamaño de problema que cabe en 1D.

La difusión jerárquica conserva esas reservas. Reduce comunicaciones, pero no elimina las copias privadas. Usar una sola copia compartida de B por nodo cambiaría el término rM por M y requeriría otra implementación, sincronización y verificación de visibilidad. Es una vía concreta para reducir memoria; su efecto temporal todavía no se ha medido.

6.3. Qué aporta y qué cuesta la malla 2D

Con 64 procesos, $q = 8$ y $b = 3072/8 = 384$. Cada proceso reserva cinco bloques $b \times b$: A_{loc} , B_{loc} , dos temporales y C_{loc} . En el servidor permanecen además las matrices completas A, B, C y

un buffer de empaquetado. Con dieciséis procesos por nodo:

$$\mathcal{M}_{0,2D} = (4 + 5/4)M = 5.25M, \quad \mathcal{M}_{h>0,2D} = 1.25M.$$

Para $N = 3072$, son aproximadamente 0.369 GiB en el servidor y 0.088 GiB por worker, frente a 1.301 GiB en el servidor 1D con 64 procesos. El ahorro de reservas es real en el código, aunque la campaña no midió RSS. Esta ventaja puede importar en problemas de mayor tamaño aun cuando el tiempo del caso medido sea peor.

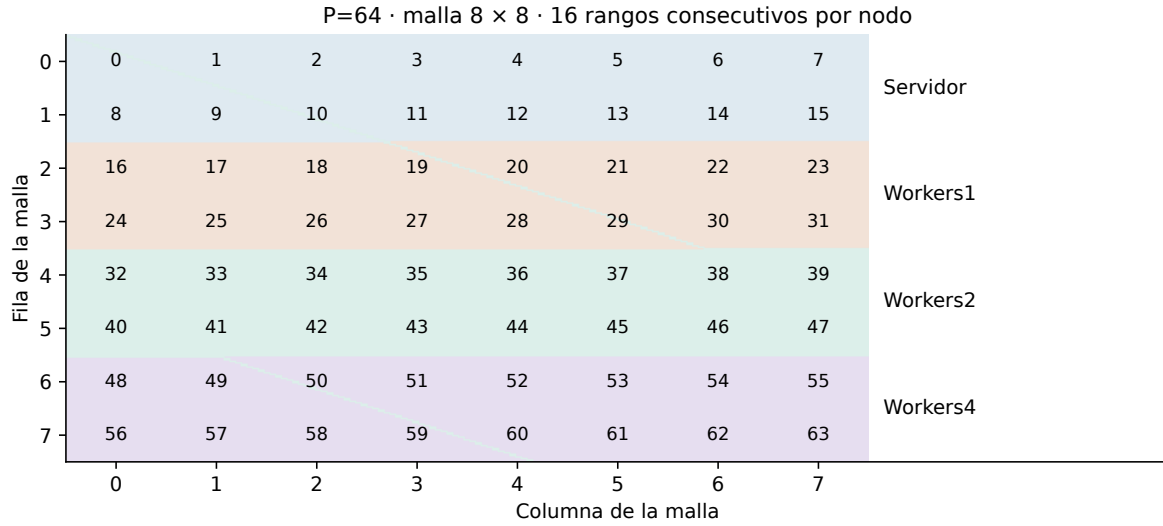


Figura 8: Asignación deducida del orden por slots: cada color es un nodo físico y cada número es un rango MPI. Dos filas completas de la malla quedan en cada equipo.

Los comunicadores de fila quedan enteramente dentro de un nodo; cada comunicador de columna incluye dos rangos de cada uno de los cuatro nodos. Por tanto, las difusiones de A por fila son locales y las de B por columna atraviesan la red. Hay ocho pasos secuenciales; en cada uno se realizan dos colectivas bloqueantes antes del cálculo. La malla lógica 8×8 no equivale a 64 computadoras ni a una red física bidimensional.

Se puede calcular una referencia ideal de carga útil entre equipos, sin cabeceras ni repeticiones de mensajes. Para 1D con jerarquía perfecta sobre cuatro nodos, distribuir A mueve $3M/4$, difundir B mueve $3M$ y reunir C mueve $3M/4$: en total $4.5M = 339.74$ MB. Para la malla 2D descrita, repartir inicialmente A y B mueve $1.5M$, difundir todos los bloques de B a los otros tres nodos suma $3M$ y reunir C añade $0.75M$: $5.25M = 396.36$ MB.

Estas cuentas suponen que cada bloque cruza una sola vez hacia cada nodo que lo necesita, con comunicación local eficiente. Son referencias de carga útil para esta colocación; no predicciones exactas de contadores. El TX observado de 678 MB para 2D supera esa referencia. La diferencia puede incluir decisiones de colectivas y tráfico auxiliar, pero no se atribuye una cantidad concreta a cada causa sin una traza. Incluso la referencia ideal explica por qué reducir la memoria por proceso no obligaba a reducir la red física en este caso: los cuatro equipos siguen necesitando todos los bloques de B , repartidos entre sus procesos.

6.4. Qué optimizar después de reducir Bcast

Tras aplicar la jerarquía, la multiplicación Ethernet aún tarda 3.217 s. El máximo mediano de cálculo es solo 0.161 s, mientras distribución y reunión siguen mostrando duraciones grandes. Hacer más rápido únicamente el bucle local difícilmente explicaría por sí solo el salto hasta 0.649

s del nodo local. No se calcula un límite de aceleración restando máximos de fases, porque esos máximos pueden corresponder a rangos distintos.

Las oportunidades siguientes afectan el camino completo: reducir copias compartiendo B , evitar repetir comunicaciones si se reutiliza la misma matriz, investigar los costes de reparto y reunión o distribuir la entrada desde su origen. Cada cambio modifica el problema operativo. Si una aplicación necesita una única matriz final en el servidor, su reunión debe seguir incluida en la métrica de decisión. Si consume resultados distribuidos, esa nueva condición se debe declarar y medir de forma separada.

7. Una corrida frente a repeticiones

Las dos baterías ejecutaron cada configuración una sola vez; la campaña repitió algunas de ellas de tres a cinco veces. Donde coinciden la prueba, el tamaño, la red y el número de procesos, se puede comparar la magnitud de las diferencias entre campañas. Como también cambió la compilación, esas diferencias no estiman por sí solas el error de una corrida única. La tabla usa la variante 1D global de la campaña y, para el trapecio, su modo simétrico.

Tabla 17: Tiempo total (s) de las corridas únicas de cada batería frente a la mediana y el rango de la campaña.

Prueba	Red	P	Corrida única		Campaña		
			Bat. 1	Bat. 2	Mediana	Rango	Reps.
Matrices $N = 3072$	Ethernet	24	0.512	0.547	0.649	0.633–0.719	5
Matrices $N = 3072$	Ethernet	96	6.175	6.148	6.260	6.233–6.266	5
Matrices $N = 3072$	Wi-Fi	24	0.553	0.553	0.666	0.581–0.722	5
Matrices $N = 3072$	Wi-Fi	96	101.042	99.945	101.082	100.293–102.991	5
Matrices $N = 6144$	Ethernet	24	–	3.839	5.279	4.998–7.343	3
Matrices $N = 6144$	Ethernet	96	–	25.037	25.194	25.065–25.322	3
Trapecio $n = 10^{11}$	Ethernet	24	3.870	3.842	5.183	4.753–5.379	3
Trapecio $n = 10^{11}$	Ethernet	96	0.964	0.975	1.012	1.004–1.024	3

7.1. Qué muestra la comparación

- **Con 96 procesos, las corridas únicas de matrices quedaron próximas a las medianas posteriores.** Con $N = 3072$ y $N = 6144$, las baterías quedan a menos de un 2 % de la mediana repetida en las redes medidas. El tiempo está dominado por la comunicación, que dio resultados próximos en los tres momentos.
- **Con 24 procesos, la campaña fue sistemáticamente más lenta:** un 19–27 % con $N = 3072$ y un 37 % con $N = 6144$. Ninguna de las corridas repetidas de $N = 3072$ por Ethernet bajó hasta los tiempos de las baterías.
- **La mayor diferencia observada está en el cálculo.** La tabla 18 separa las fases de las configuraciones con 24 procesos.

T_{Dist} cambia poco (hasta un 11 %), mientras T_{Calc} crece entre 1.3 y 1.5 veces en la campaña (0.304 a 0.455 s con $N = 3072$ por Ethernet; 2.891 a 4.447 s con $N = 6144$). La diferencia es compatible con el cambio de compilación: las baterías registran `-O3 -march=native -funroll-loops`,

Tabla 18: Máximos de cálculo y de distribución con 24 procesos: corridas únicas frente a la mediana de la campaña.

Matriz	Red	T_{Calc} máximo (s)			T_{Dist} máximo (s)		
		Bat. 1	Bat. 2	Campaña	Bat. 1	Bat. 2	Campaña
$N = 3072$	Ethernet	0.304	0.326	0.455	0.198	0.205	0.220
$N = 3072$	Wi-Fi	0.345	0.337	0.450	0.216	0.228	0.214
$N = 6144$	Ethernet	–	2.891	4.447	–	0.875	0.959

mientras la campaña registra `-O3 -std=c11 -Wall -Wextra`. No se inspeccionó el ensamblado ni se conservaron informes de vectorización que permitan atribuirlos a instrucciones concretas. También cambió el momento de ejecución. Una prueba causal requiere recompilar el mismo código, alternar binarios bajo condiciones compartidas y repetir.

El trapecio muestra un patrón similar con 24 procesos (3.84–3.87 s en las baterías frente a 5.18 s de mediana), pero ahí la comparación mezcla además dos programas: las baterías usaron `trapecio.c` y la campaña, `trapecio_asimetrico` en modo simétrico, con la corrección del extremo.

7.2. Consecuencias para la lectura del informe

1. Las conclusiones cualitativas se mantienen: en las dos baterías, un nodo con 24 procesos es el mejor punto de los valores de P ensayados para matrices y la difusión entre nodos domina el tiempo con 96 procesos.
2. Los cocientes T_{96}/T_{24} de la campaña (4.77 y 4.57) no deben mezclarse con los de las baterías (6.52 para $N = 6144$). Si mejorara únicamente el cálculo manteniendo iguales los demás costes, la configuración local, más sensible a ese cálculo, podría ganar más; los datos actuales no aíslan esa intervención.
3. Una diferencia pequeña entre dos corridas únicas con 24 procesos, por ejemplo entre redes, no es interpretable: la variación entre momentos supera esas diferencias.

8. Experimentos siguientes, ordenados por utilidad

Las propuestas de esta sección no se ejecutaron para regenerar el informe. Se ordenan por la incertidumbre que resuelven y por su relación con los costes observados. Se propone conservar un identificador único, comando expandido, versión de código y binario, reparto por nodo, afinidad, tiempos por rango, contadores y resultado numérico para cada nueva corrida. Los resultados fallidos también deben quedar registrados.

8.1. Prioridad 1: comprobar knomial en la multiplicación completa

Pregunta. ¿La selección de la colectiva reproduce la mejora de `1d-hier` sin modificar la estructura del programa?

Diseño. Mantener $N = 3072$, kernel tiled 64, $P = 96$, distribución $24 + 24 + 24 + 24$, el mismo binario y las dos redes. Comparar tres tratamientos: 1D global por defecto, 1D global con las opciones knomial y 1D jerárquica. Ejecutar al menos diez bloques con orden aleatorio y registrar la selección del componente colectivo. Es un plan de $3 \times 2 \times 10 = 60$ ejecuciones

nuevas. La etiqueta **tuned-knomial** identifica opciones solicitadas; la traza permitirá confirmar la selección efectiva.

Métricas. Tiempo total interno, duración pura del comando, coste del procedimiento, TX/RX por nodo, dispersión y huellas. Como umbral operativo propuesto, considerar útil una reducción de al menos 5 % del tiempo completo frente a la jerárquica, repetida en ambos sentidos de orden sin fallos numéricos. El 5 % expresa una preferencia práctica para el próximo estudio; no es un umbral estadístico obtenido de estos datos. Una igualdad práctica también sería informativa si evita mantener código adicional.

8.2. Prioridad 2: separar compilación y kernel local

Pregunta. ¿Cuánto de la diferencia entre campañas proviene del binario?

Diseño. A partir del mismo código, conservar binarios distintos con `-O3`, `-O3 -march=native` y `-O3 -march=native -funroll-loops`. Registrar versión del compilador, comando y SHA-256 de cada binario; alternarlos en el mismo bloque. Empezar con $N = 3072, 6144, 24$ procesos locales y cinco repeticiones: $3 \times 2 \times 5 = 30$ corridas. Mantener afinidad, tamaño de bloque, datos y estado de energía documentados. No sobrescribir los binarios que produjeron la evidencia anterior.

Métricas. Tiempo de cálculo por rango, tiempo total, informe de vectorización y, si se estudian instrucciones concretas, desensamblado del binario. GCC documenta que `-march=native` habilita extensiones de la máquina de compilación[6]; los flags por sí solos no demuestran qué instrucciones terminaron ejecutándose. Un control con BLAS optimizada puede aportar una referencia de cálculo, fijando explícitamente sus hilos para no multiplicarlos por el número de procesos MPI. Ese control necesita su propio tratamiento y comprobación numérica.

8.3. Prioridad 3: encontrar el paralelismo local adecuado

Pregunta. ¿Cuántos procesos convienen en un nodo híbrido P/E?

Diseño. Con un binario ya fijado, barrer $P = 1, 2, 4, 8, 12, 16, 20, 24$ y $N = 3072, 6144$, cinco veces por configuración: 80 corridas. Registrar rango, identificador de CPU y tipo de núcleo, y definir explícitamente cómo se selecciona el subconjunto de núcleos. Un barrido de cantidad de procesos sin controlar qué núcleos se usan mezcla dos cambios.

Métricas. Tiempo total, cálculo máximo y promedio, bytes de memoria si se dispone de contadores fiables, frecuencia y dispersión. El antecedente local de 16 procesos justifica este barrido. Se busca el mínimo observado con un mapa reproducible, no demostrar de antemano que ocho P, dieciséis procesos o todos los núcleos sean mejores.

8.4. Prioridad 4: una copia de B por nodo

Pregunta. ¿Compartir el almacenamiento reduce memoria y copias sin introducir espera mayor?

Diseño. Añadir una variante con un buffer de B compartido dentro del nodo, difusión entre líderes y sincronización conforme al modelo de memoria de MPI. Compararla con la jerárquica actual, que conserva buffers privados. Validar primero matrices pequeñas elemento por elemento; después usar los mismos tamaños y mapas de la campaña. La comparación debe mantener el mismo kernel y reportar por separado inicialización, creación del recurso compartido y región repetida.

Métricas. RSS o memoria pico por proceso y nodo, tiempo total, copias, validación y coste de sincronización. La reducción de reservas prevista en el modelo es una hipótesis comprobable. No se promete una aceleración proporcional a la reducción de memoria: NUMA, cachés y concurrencia de lectura pueden cambiar el resultado.

8.5. Prioridad 5: 2D con trazas y una interfaz de entrada explícita

Antes de ampliar la malla, medir por separado empaquetar, repartir, cada paso de difusión, calcular, reunir y desempaquetar. Registrar qué rangos participan en cada colectiva y bytes por nodo. Comparar la implementación actual con otra que reciba matrices ya distribuidas solo si esa condición representa una aplicación real; mostrar ambos tiempos, con y sin preparación centralizada.

El solapamiento mediante colectivas no bloqueantes exige doble buffer, dependencias correctas y verificar que exista progreso durante el cálculo. Cambiar únicamente a una llamada no bloqueante no garantiza solapamiento efectivo. La primera pregunta es cuánto tiempo puede ocultarse según una traza por rango. Si el beneficio esperado es menor que los costes de preparación, conviene resolver estos primero.

8.6. Prioridad 6: calibración P/E y estado de la red

Para el trapecio, medir capacidad por núcleo con la misma función y el mismo binario; verificar el mapa físico y calcular pesos a partir de duraciones conservadas. Comparar reparto simétrico, peso fijo 1.2658 y peso calibrado. Conservar tanto el máximo absoluto como el cociente de desbalance, porque este último puede mejorar sin reducir el tiempo total.

Para investigar Wi-Fi, añadir mediciones de caudal punto a punto y simultáneo, señal y contadores de reintentos o retransmisiones disponibles; registrar carga de fondo. Esas pruebas responden una pregunta distinta de la eficiencia del kernel. Los pings existentes indican conectividad y latencia de mensajes pequeños; no miden el ancho de banda de una colectiva de 75.5 MB ni prueban saturación de radio. Ejecutar sesiones en distintos horarios permitiría evaluar si las conclusiones de tráfico conservan su tamaño de efecto.

8.7. Criterio de cierre de una nueva campaña

Definir antes de ejecutar cuál comparación decide el resultado, mantener las observaciones individuales y establecer un número de bloques. Si aparecen valores extremos, registrar el estado del sistema y conservarlos; repetir por un criterio declarado, no hasta conseguir el valor deseado. Publicar mejoras y regresiones, y separar las mediciones exploratorias usadas para elegir parámetros de las usadas para confirmar el resultado. Así, el siguiente informe podrá explicar una causa con mayor precisión en lugar de acumular cifras de configuraciones incompatibles.

9. Auditoría, amenazas a la validez y procedencia

9.1. Comprobaciones repetibles

`auditar_evidencia.py` contrasta CSV, logs, contadores y manifiestos. Los tiempos y huellas se cotejan con las tolerancias declaradas en el script, compatibles con la precisión impresa; no se afirma una igualdad de valores internos no registrados. Las 118 filas de la campaña incluyen además comprobación de los deltas TX/RX conservados en el JSON de cada log. Los manifiestos se verifican mediante SHA-256.

Tabla 19: Auditoría automática de las copias disponibles al reconstruir esta edición.

Comprobación	Alcance	Resultado
Primera batería: CSV frente a logs	50 filas (T_Total, P)	sin discrepancias
Segunda batería: CSV consolidado frente a logs	75 filas (4 tiempos)	sin discrepancias
Huellas de matrices en ambas baterías	6 tamaños de N	sin discrepancias
Error del trapecio respecto de π	40 ejecuciones	máximo 1.61e-10
Campaña: CSV frente a logs	118 filas (tiempos y huellas)	sin discrepancias
Campaña: TX/RX frente a contadores del log	118 filas	sin discrepancias
Campaña: sellos SHA256SUMS	122 archivos	sin discrepancias
Copias de evidencia frente a procedencia	152 archivos	sin discrepancias
Originales externos, si están disponibles	152 archivos	distintos: graficar.py, mpi_matrix_2d.c
Compilación registrada en la campaña	entorno.json	mpicc -O3 -std=c11 -Wall -Wextra
Gráficos de bateria_expandida/... ./analisis/	metricas.csv	solo Wi-Fi

La ausencia de discrepancias significa que la información contrastada concuerda y que los archivos sellados mantienen sus hashes actuales. Un hash no prueba que la medición original fuera correcta ni establece por sí solo cuándo se hizo. La suma diagonal y ponderada aporta un control de consistencia de matrices; la igualdad de una huella entre rangos controla la difusión. Ambos son menos completos que verificar todos los elementos contra una referencia independiente.

9.2. Fuentes de sesgo o incertidumbre

Tabla 20: Amenazas a la interpretación y tratamiento utilizado.

Factor	Consecuencia y tratamiento
Una corrida en las baterías	No estimar su variabilidad interna. Presentar valores únicos y contrastarlos, sin mezclarlos, con la campaña repetida.
Tres o cinco repeticiones	Estadística descriptiva; conservar todos los puntos y no declarar estabilidad universal o significación.
Wi-Fi y switch compartidos	El estado de fondo puede cambiar. El orden aleatorio reduce un sesgo sistemático de orden, pero no controla la red.
Compilación distinta	Comparaciones causales dentro de la campaña; contraste entre campañas con advertencia explícita.
CPU híbrida y frecuencias	Procesos distintos no equivalen a capacidad idéntica. No atribuir pesos P/E sin comprobar el mapa físico.
Contadores de interfaz	Incluir control y tráfico ajeno posible; distinguir TX agregado de TX+RX del servidor.
Máximos por fase	No sumar fases ni asignar porcentajes exclusivos de tiempo ahorrado.
Entrada y salida centralizadas	Los resultados de 2D incluyen empaquetado y recogida; no extrapolar a entradas ya distribuidas.

9.3. Problemas de procedencia detectados

Fuente de matrices modificada. La copia `src/mpi_matrix_2d.c` se amplió para añadir `1d-hier` y separar `Scatterv/Bcast`. El hash en `datos/procedencia.json` había sido actualizado a esa versión y, por tanto, no identifica el programa original de las baterías. Esta edición recupera las fuentes del commit `f81f488` en `fuentes/codigo_baterias/`, junto con un manifiesto del commit, rutas y hashes. Se preserva el manifiesto anterior para no reescribir su historia. Recuperar la fuente mejora la trazabilidad; no reconstruye retroactivamente el binario exacto ejecutado.

Gráficos sobrescritos en una carpeta externa. En la copia de trabajo `experimentos_96/bateria_expandida/20260925_003457/analisis/`, el CSV de métricas corresponde solo a Wi-Fi. El generador compartía destino para ambas redes, de modo que una podía reemplazar los gráficos de la otra. Esta edición usa el CSV consolidado conservado en el repositorio para reconstruir sus figuras y tablas. No infiere la red a partir de un nombre de carpeta.

Borradores con afirmaciones no verificadas. Los documentos auxiliares incluían una mejora del trapecio basada en una sola repetición, valores de desbalance que no coinciden con los logs, comparaciones que mezclaban mínimos con medianas y una atribución de los 1112 MB a configuraciones distintas. Se conserva el material de origen como contexto, pero las cifras del informe se obtienen de registros primarios. El peso P/E exploratorio no cuenta con salidas individuales conservadas.

Originales externos y versiones de gráficos. La auditoría puede consultar la carpeta externa `experimentos_96` si está disponible. Detecta diferencias en `graficar.py` y `mpi_matrix_2d.c`; no las trata como cambios de los resultados crudos. La reconstrucción del informe no depende de esos originales. Si se traslada la carpeta autocontenida, la parte de auditoría externa puede indicar que no los encontró mientras las verificaciones internas siguen siendo reproducibles.

Antecedentes con respaldo parcial. Las series históricas de caché y tráfico se conservan principalmente en JSON; el caso de 906.351 s tiene además log y script. Su trazabilidad es menor que la campaña con logs por corrida y hashes. Se mantienen como antecedentes y no se usan para atribuir causalmente a una sola opción toda la diferencia de tiempo entre etapas.

10. Conclusiones

10.1. Respuesta al problema de rendimiento

Los datos respaldan que el límite principal de las matrices distribuidas es el coste de mover y coordinar datos entre nodos en las configuraciones probadas. La evidencia más fuerte no es el cociente entre la corrida histórica de 906 s y una nueva: es la comparación controlada entre 1D global y jerárquica, con el mismo tamaño, número de procesos, kernel y binario. Con $N = 3072$ y 96 procesos, la jerarquía redujo el tiempo interno aproximadamente un 49 % y el TX agregado algo más de un 52 % en ambas redes. La dirección del resultado se conserva en las cinco repeticiones.

Aun con esa mejora, cuatro nodos no superaron a un nodo de 24 procesos. En las dos baterías, 24 fue el mejor punto entre 1, 24, 48, 72, 96; en la campaña, 1D global con 24 fue más rápido que con 96 hasta $N = 7168$. Esto no determina el número óptimo dentro del nodo: el antecedente de 16 procesos y las diferencias de compilación justifican un barrido local nuevo. Tampoco determina un tamaño de cruce no medido ni el comportamiento de la jerárquica con matrices mayores.

El trapecio grande representa un uso más favorable del clúster. Las baterías alcanzaron aceleraciones internas cercanas a 68 con 96 procesos por Ethernet, gracias a mucho cálculo independiente y pocos datos finales. El caso pequeño muestra el límite contrario: la coordinación y el lanzamiento pueden dominar. Un buen tiempo interno no basta para elegir una configuración si el usuario ejecuta operaciones aisladas y debe esperar todo el procedimiento.

10.2. Qué optimizaciones quedan respaldadas y cuáles abiertas

La difusión jerárquica es una mejora comprobada para la multiplicación distribuida estudiada. La selección knomial es una alternativa prometedora en el microbenchmark: obtuvo TX agregado similar y tiempos cerca de un 3 % menores que la jerárquica, aunque con la transmisión concentrada en el servidor. La prueba decisiva pendiente es la multiplicación completa con 96 procesos y trazas de selección de la colectiva.

SUMMA 2D disminuye las reservas explícitas por proceso, pero en la configuración $P = 64$, malla 8×8 , aumentó tiempo y TX. La colocación de dos filas por nodo, la entrada centralizada, el empaquetamiento y las colectivas bloqueantes delimitan ese resultado. Memoria, volumen de interfaz y tiempo son objetivos relacionados, pero no intercambiables. El reparto ponderado del trapecio tampoco mostró una ganancia consistente; su menor cociente de desbalance en algunas corridas no implicó un menor tiempo total.

10.3. Alcance de la explicación y de la reproducibilidad

La corrida de 906.351 s queda identificada sin ambigüedad: ocho procesos, dos en cada uno de cuatro nodos, matriz de 3072×3072 , Wi-Fi y ejecutable histórico. Su repetición de 322.513 s impide tratar ese único valor extremo como una base estable para cuantificar toda la mejora posterior. La documentación de Open MPI y los comandos conservados tampoco permiten afirmar que antes faltara memoria compartida local o que un único parámetro explique el cambio completo.

El informe conserva 243 registros principales, las series históricas y la cadena desde cada tabla hasta CSV y log. Se verificaron coherencia numérica impresa y manifiestos, con las limitaciones declaradas. La nueva edición aporta análisis y modelos derivados de esa evidencia; no convierte hipótesis en mediciones. El siguiente paso con mayor utilidad es comprobar la selección de Bcast en la multiplicación completa y aislar el efecto de compilación antes de ampliar tamaños o rediseñar kernels.

Recomendación basada en lo medido. Para estas multiplicaciones aisladas y los tamaños ensayados, comenzar por la configuración local y optimizarla con un barrido controlado. Cuando sea necesario distribuir el cálculo, usar la difusión jerárquica como referencia validada y medir el coste completo. Para cargas independientes como el trapecio grande, el clúster ya ofrece una aceleración sustancial, cuya utilidad práctica depende de cuánto lanzamiento y coordinación se pueda amortizar.

A. Reproducción, archivos y trazabilidad

A.1. Carpeta autocontenida y reconstrucción

El repositorio contiene evidencia, código, scripts de análisis, fuentes LaTeX y figuras. No necesita que los workers estén conectados para reconstruir el informe. La carpeta usada en esta computadora es `/home/alumno16/cluster-mpi-96-optimizaciones`. Los comandos siguientes regeneran tablas y figuras desde las copias locales y compilan el documento:

```
1 cd /home/alumno16/cluster-mpi-96-optimizaciones/revision
2 ./compilar.sh
```

Se requiere Python 3 con NumPy y Matplotlib, `pdflatex` y los paquetes LaTeX indicados en el preámbulo. `compilar.sh` ejecuta cuatro scripts: `generar_datos.py`, `generar_optimizacion.py`, `auditar_evidencia.py` y `generar_analisis.py`; después hace tres pasadas de LaTeX. Los scripts leen la evidencia sin editarla. `EVIDENCIA_MPI=/ruta/al/repositorio` permite indicar otra copia. Las comprobaciones contra originales externos son opcionales y su número puede variar entre computadoras; las tablas de rendimiento proceden de los datos internos.

Tabla 21: Mapa de archivos. Rutas relativas a la raíz del repositorio.

Archivo o directorio	Función
<code>datos/ethernet/</code> , <code>datos/wifi/</code>	CSV y logs de las 50 ejecuciones iniciales.
<code>datos/ampliada/</code>	CSV, JSON y logs de la batería de 75 ejecuciones.
<code>datos/historicos/</code>	Series de madrugada, mediodía y kernel local.
<code>resultados_optimizacion/20260925_074354/</code>	82 ejecuciones: matrices, Bcast y 2D.
<code>resultados_optimizacion/20260925_extra/</code>	36 ejecuciones: tamaños mayores y trapecio.
<code>src/</code> , <code>experimentos.py</code>	Programa y protocolo de la campaña.
<code>fuentes/codigo_baterias/</code>	Fuentes recuperadas del commit histórico, con manifestación de origen.
<code>revision/datos/analisis_profundo.json</code>	Efectos sin redondeo nuevo, razones por bloque, tráfico por nodo y modelo de memoria.
<code>revision/datos/catalogo_campana.csv</code>	Las 118 observaciones con hora de inicio, reparto, métricas y ruta a su log.
<code>revision/datos/auditoria.json</code>	Comprobaciones automáticas de evidencia.
<code>revision/estudio_*.tex</code>	Metodología, resultados, modelos, propuestas y anexos de esta edición.

A.2. Cómo seguir una cifra hasta su ejecución

La reducción Ethernet de 6.260 a 3.217 s se reconstruye filtrando el CSV de la campaña principal por `kind=matriz`, `red=Ethernet`, `n=3072`, `p=96`, `nodos=4`, y separando `variante=1d` de `1d-hier`. Cada grupo tiene cinco filas. Se toma la mediana de `T_Total`, sin mezclar la columna `pared_s`. El campo `log` apunta al archivo que conserva el comando, los contadores y la línea `RESULT_2D`.

Para TX por nodo se resta `antes[host][0]` a `despues[host][0]` en el objeto JSON inicial del log y se divide entre 10^6 . Se resume cada host por separado. La mediana agregada se calcula sumando primero los cuatro deltas de cada corrida; ambas operaciones no se intercambian sin cambiar el estadístico.

En las primeras baterías, el nombre del log contiene experimento, tamaño y procesos; en la ampliada añade la red. Los CSV originales conservan la precisión exportada. La impresión

con tres decimales en una tabla no debe usarse para recalcular un efecto pequeño: el JSON de análisis conserva los valores usados antes del redondeo.

A.3. Comando representativo de la campaña

El siguiente ejemplo documenta $24+24+24+24$ por Ethernet. La ruta de binarios corresponde al despliegue de la campaña. El comando exacto de cada ejecución se encuentra en su log; reconstruir el PDF no ejecuta estos comandos.

```
1 mpirun \
2   -H 10.7.50.202:24,10.7.50.203:24,10.7.50.201:24,10.7.50.204:24 \
3   -np 96 --map-by slot --rank-by slot --bind-to core \
4   --mca plm_rsh_agent "ssh -F scripts/ssh_config" \
5   --mca btl self,vader,tcp \
6   --mca btl_tcp_if_include 10.7.50.0/24 \
7   --mca oob_tcp_if_include 10.7.50.0/24 \
8   /var/tmp/cluster-mpi-96-optimizaciones/bin/mpi_matrix_2d \
9   3072 1d-hier tiled 64
```

Para conocer el plan conservado sin ejecutar pruebas:

```
1 cd /home/alumno16/cluster-mpi-96-optimizaciones
2 python3 experimentos.py plan
3 python3 experimentos.py plan-extra
```

Los comandos históricos de `scripts/` y los programas actuales corresponden a etapas distintas. Compilar hoy una fuente recuperada reconstruye un programa, pero no garantiza repetir su tiempo: compilador, flags, frecuencia, carga y red forman parte de la configuración.

B. Detalle de las baterías y antecedentes

B.1. Primera batería: trapecio

Cada tabla conserva las dos redes y los cinco números de procesos. Son ejecuciones únicas. T_E y T_W están en segundos; speedup y eficiencia usan el proceso único de la misma red.

Tabla 22: Trapecio, $n = 10^8$.

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	0.066698	0.067350	1.00	1.00	1.01
24	0.003926	0.005561	16.99	12.11	1.42
48	0.002061	0.018343	32.36	3.67	8.90
72	0.001983	0.089785	33.63	0.75	45.28
96	0.001661	0.146423	40.16	0.46	88.15

Tabla 23: Trapecio, $n = 10^{11}$.

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	66.113517	66.526926	1.00	1.00	1.01
24	3.869576	4.045816	17.09	16.44	1.05
48	1.927789	2.016062	34.29	33.00	1.05
72	1.285578	1.368964	51.43	48.60	1.06
96	0.964298	1.053217	68.56	63.17	1.09

B.2. Primera batería: matrices y controles

La tabla de $N = 3072$ se encuentra en el cuerpo del informe; las dos siguientes completan las 30 ejecuciones de matrices de la primera batería.

Tabla 24: Matrices, $N = 1024$.

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	0.145456	0.147472	1.00	1.00	1.01
24	0.032678	0.034179	4.45	4.31	1.05
48	0.347345	5.549577	0.42	0.03	15.98
72	0.567399	9.269884	0.26	0.02	16.34
96	0.772748	14.113333	0.19	0.01	18.26

Tabla 25: Matrices, $N = 2048$.

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	1.199625	1.244563	1.00	1.00	1.04
24	0.193562	0.199079	6.20	6.25	1.03
48	1.252217	21.085757	0.96	0.06	16.84
72	1.982867	31.687304	0.60	0.04	15.98
96	2.788212	46.322868	0.43	0.03	16.61

Tabla 26: Huellas impresas en las matrices de la primera batería. Su coincidencia es un control de consistencia, no una comparación completa elemento por elemento.

N	Suma diagonal	Suma ponderada	Coinciden
1024	2453667.02	1.758790 e+10	10/10
2048	9814670.59	1.407031 e+11	10/10
3072	22083010.24	4.748730 e+11	10/10

B.3. Batería ampliada completa

Se conservan los puntos intermedios de 48 y 72 procesos. En trapecio, la columna que el CSV llama `T_Gather` corresponde a reducción; en matrices, a reunión. Distribuir vale cero cuando esa fase no existe en el programa. Los valores MB son del servidor, con el alcance explicado en metodología.

Tabla 27: Las 75 ejecuciones de la batería ampliada. Tiempos en segundos; MB = TX+RX del servidor.

Red	Prueba	Tamaño	P	Total	Distribuir	Calcular	Reunir/reducir	MB
Ethernet	trapecio	100000000	1	0.066984	0.000000	0.066982	0.000002	0.00
Ethernet	trapecio	100000000	24	0.003921	0.000000	0.003888	0.000764	0.00
Ethernet	trapecio	100000000	48	0.002026	0.000000	0.001997	0.000288	0.09
Ethernet	trapecio	100000000	72	0.002225	0.000000	0.001300	0.001144	0.19
Ethernet	trapecio	100000000	96	0.001508	0.000000	0.001065	0.000569	0.61
Ethernet	trapecio	100000000000	1	66.488999	0.000000	66.488994	0.000004	0.84
Ethernet	trapecio	100000000000	24	3.842393	0.000000	3.842303	0.651409	0.00
Ethernet	trapecio	100000000000	48	1.931728	0.000000	1.931683	0.291633	0.11
Ethernet	trapecio	100000000000	72	1.325488	0.000000	1.325426	0.219324	0.19
Ethernet	trapecio	100000000000	96	0.974949	0.000000	0.974707	0.160998	0.21
Ethernet	matrices	512	1	0.018799	0.000796	0.017238	0.000764	0.00
Ethernet	matrices	512	24	0.007499	0.005361	0.001735	0.002547	0.00
Ethernet	matrices	512	48	0.094404	0.089877	0.000978	0.039279	8.89
Ethernet	matrices	512	72	0.163042	0.152524	0.001064	0.082164	14.25
Ethernet	matrices	512	96	0.210618	0.207587	0.000859	0.097131	14.55
Ethernet	matrices	1024	1	0.145072	0.003460	0.137954	0.003657	0.00
Ethernet	matrices	1024	24	0.033787	0.023597	0.010631	0.007351	0.00
Ethernet	matrices	1024	48	0.344563	0.282172	0.006888	0.186981	35.33
Ethernet	matrices	1024	72	0.559754	0.477336	0.007298	0.314198	56.08
Ethernet	matrices	1024	96	0.760043	0.724805	0.008396	0.325031	57.63
Ethernet	matrices	2048	1	1.198498	0.014357	1.170489	0.013650	0.01
Ethernet	matrices	2048	24	0.200081	0.090880	0.116177	0.028172	0.00
Ethernet	matrices	2048	48	1.261592	1.057031	0.050448	0.806665	141.11
Ethernet	matrices	2048	72	1.985896	1.737039	0.044803	1.377896	223.46
Ethernet	matrices	2048	96	2.786657	2.697338	0.039427	1.380725	229.39
Ethernet	matrices	3072	1	3.889376	0.030689	3.828174	0.030511	0.00
Ethernet	matrices	3072	24	0.547426	0.204794	0.325547	0.080399	0.57
Ethernet	matrices	3072	48	2.799523	2.357572	0.140443	1.848716	318.25
Ethernet	matrices	3072	72	4.334296	3.833933	0.117996	3.147109	503.01
Ethernet	matrices	3072	96	6.148198	5.975176	0.103672	3.140721	516.72
Ethernet	matrices	4096	1	9.913873	0.054371	9.805386	0.054115	0.04
Ethernet	matrices	4096	24	1.324593	0.365419	1.005109	0.151139	0.03
Ethernet	matrices	4096	48	5.244585	4.244254	0.485275	3.285006	564.25
Ethernet	matrices	4096	72	7.764043	6.827211	0.311361	5.668882	894.57
Ethernet	matrices	4096	96	11.041805	10.714442	0.278325	5.773721	917.87
Ethernet	matrices	6144	1	32.887841	0.120621	32.647184	0.120034	0.06
Ethernet	matrices	6144	24	3.839080	0.874517	2.890923	0.429950	0.01

Red	Prueba	Tamaño	P	Total	Distribuir	Calcular	Reunir/reducir	MB
Ethernet	matrices	6144	48	12.016472	9.606223	1.433817	7.480491	1271.86
Ethernet	matrices	6144	72	18.010841	15.587750	1.154965	12.795243	2010.74
Ethernet	matrices	6144	96	25.036958	24.136302	0.852913	12.863855	2065.61
Wi-Fi	trapecio	100000000	1	0.067826	0.000000	0.067824	0.000002	0.00
Wi-Fi	trapecio	100000000	24	0.005612	0.000000	0.005560	0.002374	0.00
Wi-Fi	trapecio	100000000	48	0.022309	0.000000	0.001906	0.020403	0.09
Wi-Fi	trapecio	100000000	72	0.081375	0.000000	0.001301	0.080332	0.20
Wi-Fi	trapecio	100000000	96	0.115863	0.000000	0.000971	0.115080	0.23
Wi-Fi	trapecio	100000000000	1	66.857550	0.000000	66.857545	0.000005	0.00
Wi-Fi	trapecio	100000000000	24	3.937623	0.000000	3.937528	0.750703	0.00
Wi-Fi	trapecio	100000000000	48	1.942522	0.000000	1.942484	0.324062	0.09
Wi-Fi	trapecio	100000000000	72	1.404793	0.000000	1.305065	0.323523	0.20
Wi-Fi	trapecio	100000000000	96	1.135211	0.000000	0.995231	0.331233	0.23
Wi-Fi	matrices	512	1	0.018844	0.000827	0.017202	0.000815	0.00
Wi-Fi	matrices	512	24	0.007188	0.005501	0.001266	0.002235	0.00
Wi-Fi	matrices	512	48	1.557343	1.474399	0.000955	0.514640	8.59
Wi-Fi	matrices	512	72	2.159039	2.114078	0.002307	1.482042	13.71
Wi-Fi	matrices	512	96	4.299395	4.235851	0.000885	1.603827	14.06
Wi-Fi	matrices	1024	1	0.150240	0.003641	0.142939	0.003659	0.00
Wi-Fi	matrices	1024	24	0.034809	0.022568	0.012270	0.006482	0.00
Wi-Fi	matrices	1024	48	5.636130	4.556819	0.007671	3.279832	34.11
Wi-Fi	matrices	1024	72	9.071514	7.455247	0.005949	5.226155	54.13
Wi-Fi	matrices	1024	96	13.648012	11.444690	0.005462	6.329169	55.68
Wi-Fi	matrices	2048	1	1.223764	0.014336	1.194770	0.014656	0.00
Wi-Fi	matrices	2048	24	0.214929	0.101730	0.109531	0.037026	0.00
Wi-Fi	matrices	2048	48	21.343125	18.060784	0.064743	13.594398	136.27
Wi-Fi	matrices	2048	72	32.642679	27.933491	0.047720	21.664117	215.71
Wi-Fi	matrices	2048	96	47.759433	45.175900	0.038585	23.660544	221.68
Wi-Fi	matrices	3072	1	3.875501	0.032337	3.811637	0.031525	0.00
Wi-Fi	matrices	3072	24	0.552753	0.227656	0.336863	0.072620	0.00
Wi-Fi	matrices	3072	48	41.795233	35.599044	0.171494	29.535603	306.48
Wi-Fi	matrices	3072	72	67.120577	58.042856	0.143291	48.781947	485.80
Wi-Fi	matrices	3072	96	99.944942	94.994963	0.139207	55.029190	500.88
Wi-Fi	matrices	4096	1	9.576560	0.056558	9.466124	0.053877	0.00
Wi-Fi	matrices	4096	24	1.410032	0.676061	0.970629	0.277196	0.00
Wi-Fi	matrices	4096	48	73.451014	63.754598	0.520423	52.474924	543.73
Wi-Fi	matrices	4096	72	117.954422	103.268838	0.328112	88.079383	867.09
Wi-Fi	matrices	4096	96	178.957201	171.849302	0.299795	104.412173	887.70

B.4. Tráfico histórico y serie del mediodía

Estas series mantienen sus protocolos de origen. El antecedente de medianoche con ocho procesos no comparte fila con los MB de la serie de dieciséis procesos. Los JSON históricos ofrecen un respaldo más limitado que la campaña con logs completos y sellos.

Tabla 28: Serie histórica con contadores: cuatro nodos y 16 procesos en las pruebas distribuidas.

P	Versión	Rep.	Total (s)	MB TX	MB/s*
4	1D-ikj	1	55.842	342.34	6.13
4	1D-ikj	2	58.540	342.65	5.85
4	2D-tiled	1	85.681	323.87	3.78
4	2D-tiled	2	61.799	324.43	5.25
16	1D-ikj	1	338.847	1112.24	3.28
16	1D-ikj	2	322.789	1111.50	3.44
16	2D-tiled	1	179.684	423.41	2.36
16	2D-tiled	2	66.172	422.35	6.38

Tabla 29: Serie histórica del mediodía: comparación de redes y descomposiciones. Las repeticiones indicadas pertenecen a esta serie.

P	Versión	Cable (s)	Cable MB	Wi-Fi (s)	Wi-Fi MB
4	1D-ikj	4.388	357.31	56.154	342.02
4	1D-tiled	3.713	357.27	70.691	342.15
4	2D-tiled	3.170	337.75	51.559	323.30
16	1D-ikj	6.610	1152.07	198.721	1120.43
16	1D-tiled	5.467	1151.70	188.147	1125.16
16	2D-tiled	3.550	437.06	91.905	423.05
64	1D-ikj	4.602	357.91	–	–
64	1D-tiled	3.153	357.79	–	–
64	2D-tiled	5.592	677.95	–	–

C. Detalle de la campaña repetida

La tabla siguiente resume todas las configuraciones de las 118 ejecuciones. El reparto por nodo se obtiene de P/H y de la tabla 4. Para Bcast se usa `T_Bcast`; para las otras pruebas, `T_Total`. Los valores individuales están en `revision/datos/catalogo_campana.csv` y las comparaciones centrales se muestran como puntos en las figuras. Las estadísticas no incorporan pilotos.

Tabla 30: Campaña de optimización: medianas de las corridas válidas, con mínimo y máximo. TX suma la transmisión de las interfaces de los nodos participantes.

Red	Prueba	Tamaño	P	Nodos	Variante	Reps.	T med. (s)	Rango (s)	TX (MB)
Ethernet	bcast	3072	16	1	hier	3	0.124	0.117–0.129	0.0
Ethernet	bcast	3072	16	1	world	3	0.116	0.116–0.127	0.0
Ethernet	bcast	3072	16	4	hier	3	2.034	2.030–2.035	238.4
Ethernet	bcast	3072	16	4	tuned-knomial	3	1.977	1.974–1.983	238.4
Ethernet	bcast	3072	16	4	tuned-scatter	3	4.372	4.010–4.374	1032.4
Ethernet	bcast	3072	16	4	world	3	4.294	4.003–4.351	1032.5
Wi-Fi	bcast	3072	16	4	hier	3	30.464	29.791–31.940	228.0
Wi-Fi	bcast	3072	16	4	tuned-knomial	3	29.575	29.354–29.650	228.6
Wi-Fi	bcast	3072	16	4	tuned-scatter	3	134.479	132.263–134.957	1006.5
Wi-Fi	bcast	3072	16	4	world	3	133.043	131.304–133.633	1006.5
Ethernet	matriz	3072	24	1	1d	5	0.649	0.633–0.719	0.0
Ethernet	matriz	3072	24	1	1d-hier	5	0.691	0.677–0.799	0.0
Ethernet	matriz	3072	64	4	1d	3	3.147	3.147–3.148	358.0
Ethernet	matriz	3072	64	4	2d	3	5.492	5.469–5.546	678.1
Ethernet	matriz	3072	96	4	1d	5	6.260	6.233–6.266	755.5
Ethernet	matriz	3072	96	4	1d-hier	5	3.217	3.088–3.288	358.9
Ethernet	matriz	6144	24	1	1d	3	5.279	4.998–7.343	0.6
Ethernet	matriz	6144	96	4	1d	3	25.194	25.065–25.322	3022.4
Ethernet	matriz	7168	24	1	1d	3	7.686	7.561–7.703	0.4
Ethernet	matriz	7168	96	4	1d	3	35.116	35.086–35.130	4113.9
Wi-Fi	matriz	3072	24	1	1d	5	0.666	0.581–0.722	0.0
Wi-Fi	matriz	3072	24	1	1d-hier	5	0.673	0.608–4.769	0.0
Wi-Fi	matriz	3072	64	4	1d	3	51.728	48.975–52.122	343.0
Wi-Fi	matriz	3072	64	4	2d	3	87.314	86.305–87.779	677.7
Wi-Fi	matriz	3072	96	4	1d	5	101.082	100.293–102.991	729.3
Wi-Fi	matriz	3072	96	4	1d-hier	5	51.479	50.817–52.659	342.9
Ethernet	trapecio	10^8	24	1	asimetrico	3	0.004	0.004–0.007	0.0
Ethernet	trapecio	10^8	24	1	simetrico	3	0.007	0.005–0.008	0.0
Ethernet	trapecio	10^8	96	4	asimetrico	3	0.002	0.002–0.002	0.6
Ethernet	trapecio	10^8	96	4	simetrico	3	0.002	0.001–0.006	0.4
Ethernet	trapecio	10^{11}	24	1	asimetrico	3	4.940	4.528–5.153	0.1
Ethernet	trapecio	10^{11}	24	1	simetrico	3	5.183	4.753–5.379	0.0
Ethernet	trapecio	10^{11}	96	4	asimetrico	3	1.046	1.026–1.198	0.5
Ethernet	trapecio	10^{11}	96	4	simetrico	3	1.012	1.004–1.024	0.8

Tabla 31: Razones entre ejecuciones del mismo bloque aleatorizado. A/B coincide con la tabla 12; no se exige que las corridas de un bloque sean consecutivas.

Red y P	A / B	Mínimo $T_{A,j}/T_{B,j}$	Mediana	Máximo
Ethernet 96	1d / 1d-hier	1.906	1.946	2.027
Ethernet 16	world / hier	1.972	2.110	2.139
Ethernet 16	hier / tuned-knomial	1.026	1.029	1.029
Ethernet 64	2d / 1d	1.738	1.744	1.762
Wi-Fi 96	1d / 1d-hier	1.953	1.956	2.010
Wi-Fi 16	world / hier	4.165	4.310	4.486
Wi-Fi 16	hier / tuned-knomial	1.015	1.030	1.077
Wi-Fi 64	2d / 1d	1.675	1.697	1.762

D. Referencias

- [1] Ian Foster. *Designing and Building Parallel Programs*. Addison-Wesley, 1995. Capítulo 2, Designing Parallel Algorithms. <https://www.mcs.anl.gov/~itf/dbpp/text/node14.html>.
- [2] Robert A. van de Geijn y Jerrell Watts. *SUMMA: Scalable Universal Matrix Multiplication Algorithm*. LAPACK Working Note 96. <https://www.netlib.org/lapack/lawnspdf/lawn96.pdf>.
- [3] Open MPI. *FAQ: Tuning the run-time characteristics of MPI TCP communications*. Documentación para Open MPI 4.x y anteriores. <https://www.open-mpi.org/faq/?category=tcp>.
- [4] Open MPI 4.1.6. *coll_tuned_bcast_decision.c*. Enumeración y selección de algoritmos de difusión. https://github.com/open-mpi/ompi/blob/v4.1.6/ompi/mca/coll/tuned/coll_tuned_bcast_decision.c.
- [5] Open MPI. *MPI_Comm_split_type*. Se usa la semántica de MPI_COMM_TYPE_SHARED, no las extensiones posteriores a la versión del laboratorio. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Comm_split_type.3.html.
- [6] GNU Compiler Collection. *x86 Options*. Opciones `-march` y selección de arquitectura. <https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html>.
- [7] Registros y fuentes del laboratorio, 24–25 de septiembre de 2026. CSV, JSON, logs, manifiestos y código conservados en <https://github.com/dtamotu/cluster-mpi-96-optimizaciones>. La copia local permite regenerar el informe sin acceder al repositorio remoto.